HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN

BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: IOT VÀ ỨNG DỤNG
MÃ HỌC PHẦN: INT14149

ĐỀ TÀI
HỆ THỐNG GIÁM SÁT VÀ ĐIỀU KHIỂN MÔI TRƯỜNG PHÒNG DỰA TRÊN IOT

Sinh viên thực hiện:

B23DCAT011 Lưu Đức Anh

Giảng viên hướng dẫn: TS. Nguyễn Quốc Uy

HỌC KỲ 1 NĂM HỌC 2026-2027

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 7
1.1 Bối cảnh 7
1.2 Lý do chọn đề tài 7
1.3 Mục tiêu 7
1.4 Phạm vi đề tài 8
1.5 Yêu cầu chức năng 8
1.6 Yêu cầu phi chức năng 9
1.7 Bố cục báo cáo 9
CHƯƠNG 2. KIẾN TRÚC HỆ THỐNG 10
2.1 Kiến trúc tổng thể 10
2.2 Kiến trúc phần cứng 10
2.2.1 Danh sách linh kiện 10
2.2.2 Sơ đồ đấu nối 11
2.3 Kiến trúc phần mềm 11
2.3.1 Phân rã tiến trình phía Backend 11
2.3.2 Giao thức truyền thông MQTT 12
2.3.3 Luồng dữ liệu chính 13
2.4 Kiến trúc tổng quan use case 13
2.4.1 Tác nhân 13
2.4.2 Biểu đồ use case tổng quát 14
2.4.3 Quan hệ giữa các use case 15
2.4.4 Quy tắc nghiệp vụ 15
CHƯƠNG 3. THIẾT KẾ CHI TIẾT VÀ CÔNG NGHỆ SỬ DỤNG 16
3.1 Công nghệ sử dụng 16
3.2 Thiết kế cơ sở dữ liệu 16
3.2.1 Bảng danh mục cảm biến (sensors_sensordevice) 17
3.2.2 Bảng số đo (sensors_sensordata) 18
3.2.3 Bảng thiết bị chấp hành (devices_device) 18
3.2.4 Bảng người dùng (users_user) 19
3.2.5 Bảng lịch sử thao tác (devices_actionhistory) 19
3.2.6 Chỉ mục 20
3.3 Thiết kế Backend 20

3.3.1 Cấu trúc mã nguồn 20
3.3.2 Xử lý lệnh điều khiển 21
3.3.3 MQTT worker 22
3.3.4 Xác thực và thời gian thực 23
3.4 Thiết kế Frontend 25
3.4.1 Cấu trúc mã nguồn 25
3.4.2 Các màn hình 25
3.4.3 Các cơ chế xử lý chính 26
3.5 Thiết kế API 26
3.5.1 Quy ước chung 26
3.5.2 Danh sách endpoint 27
3.5.3 Tham số lọc 27
3.5.4 Mã lỗi 27
3.6 Thiết kế Firmware 28
CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM 30
4.1 Môi trường triển khai 30
4.2 Kết quả phần cứng 30
4.3 Kết quả giao diện 32
4.4 Kiểm thử API bằng Postman 35
4.5 Kết quả kiểm thử 37
4.6 Các lỗi phát hiện và khắc phục 38
CHƯƠNG 5. KẾT LUẬN 40
5.1 Kết quả đạt được 40
5.2 Hạn chế 40
5.3 Hướng phát triển 40
TÀI LIỆU THAM KHẢO 42

DANH MỤC CÁC HÌNH VẼ

Hình 1. Kiến trúc tổng thể hệ thống 10
Hình 2. Sơ đồ đấu nối phần cứng 11
Hình 3. Phân rã tiến trình phía Backend 12
Hình 4. Biểu đồ use case tổng quát 14
Hình 5. Sơ đồ thực thể liên kết (ERD) 17
Hình 6. Biểu đồ tuần tự điều khiển thiết bị (mức nghiệp vụ) 21
Hình 7. Biểu đồ tuần tự điều khiển thiết bị (mức triển khai) 22
Hình 8. Biểu đồ tuần tự thu thập và truyền số liệu cảm biến 23
Hình 9. Biểu đồ tuần tự phát hiện lệnh quá thời gian chờ 23
Hình 10. Biểu đồ tuần tự mở Dashboard và thiết lập kênh thời gian thực 24
Hình 11. Thiết kế giao diện trên Figma 26
Hình 12. Biểu đồ tuần tự tra cứu lịch sử có lọc và phân trang 28
Hình 13. Mạch phần cứng sau khi lắp ráp 30
Hình 14. Đèn và quạt (LED) được bật từ giao diện web 31
Hình 15. Nhật ký Serial Monitor của ESP8266 31
Hình 16. Màn hình đăng nhập 32
Hình 17. Màn hình Dashboard 32
Hình 18. Công tắc đang chờ phản hồi từ thiết bị 32
Hình 19. Dashboard khi thiết bị ngoại tuyến 33
Hình 20. Màn hình Data Sensor 33
Hình 21. Màn hình Action History 34
Hình 22. Màn hình Profile 34
Hình 23. Tài liệu API trên Swagger UI 34
Hình 24. Đăng nhập và nhận token (POST /api/auth/login) 35
Hình 25. Số đo mới nhất của từng cảm biến (GET /api/sensors/latest) 35
Hình 26. Tìm số đo nhiệt độ đúng bằng 27.5 °C, có phân trang (GET /api/sensors) 36
Hình 27. Gửi lệnh bật đèn, máy chủ trả 202 Accepted với trạng thái PENDING (POST /api/devices/1/control) 36
Hình 28. Lịch sử thao tác: lệnh bật đèn ở hình trước bị đánh FAILED do không có phản hồi từ thiết bị trong 5 giây (GET /api/actions) 37
Hình 29. Gọi API khi chưa đăng nhập bị từ chối với mã 401 UNAUTHENTICATED 37

DANH MỤC CÁC BẢNG BIỂU

Bảng 1. Mục tiêu của đề tài 7
Bảng 2. Yêu cầu chức năng 8
Bảng 3. Yêu cầu phi chức năng 9
Bảng 4. Danh sách linh kiện 10
Bảng 5. Đấu nối chân ESP8266 11
Bảng 6. Hai tiến trình phía Backend 11
Bảng 7. Ba topic MQTT của hệ thống 12
Bảng 8. Các tác nhân của hệ thống 13
Bảng 9. Danh sách use case 14
Bảng 10. Quan hệ giữa các use case 15
Bảng 11. Quy tắc nghiệp vụ 15
Bảng 12. Công nghệ sử dụng 16
Bảng 13. Các bảng dữ liệu nghiệp vụ 17
Bảng 14. Cấu trúc bảng danh mục cảm biến 17
Bảng 15. Cấu trúc bảng số đo 18
Bảng 16. Cấu trúc bảng thiết bị chấp hành 18
Bảng 17. Cấu trúc bảng người dùng 19
Bảng 18. Cấu trúc bảng lịch sử thao tác 19
Bảng 19. Các chỉ mục 20
Bảng 20. Các ứng dụng Django của Backend 20
Bảng 21. Vai trò các file trong mỗi ứng dụng 20
Bảng 22. Cấu trúc mã nguồn Frontend 25
Bảng 23. Chức năng các màn hình 25
Bảng 24. Quy ước chung của API 26
Bảng 25. Danh sách endpoint 27
Bảng 26. Tham số lọc của hai bảng lịch sử 27
Bảng 27. Mã lỗi của API 27
Bảng 28. Môi trường triển khai 30
Bảng 29. Kết quả kiểm thử 37
Bảng 30. Kết quả đo hiệu năng 38
Bảng 31. Các lỗi phát hiện và cách khắc phục 38
Bảng 32. Hạn chế của hệ thống 40
Bảng 33. Hướng phát triển 40

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1 Bối cảnh

Việc theo dõi các thông số môi trường của một căn phòng (nhiệt độ, độ ẩm, cường độ ánh sáng) và điều khiển các thiết bị điện trong phòng là nhu cầu phổ biến ở nhà thông minh, phòng máy chủ, phòng thí nghiệm hay kho bảo quản. Cách làm thủ công đòi hỏi người dùng phải có mặt tại chỗ để đọc chỉ số và bật/tắt thiết bị, không lưu lại được diễn biến theo thời gian và không thể can thiệp từ xa.

Trong vài năm gần đây, các vi điều khiển giá rẻ tích hợp sẵn WiFi như ESP8266, cùng với giao thức truyền thông nhẹ MQTT được thiết kế riêng cho thiết bị tài nguyên hạn chế, đã khiến việc xây dựng một hệ thống giám sát và điều khiển hoàn chỉnh trở nên khả thi với chi phí phần cứng dưới 200.000 đồng.

1.2 Lý do chọn đề tài

Đề tài "Hệ thống giám sát và điều khiển môi trường phòng dựa trên IoT" được lựa chọn vì những lý do sau:

- **Tính thực tiễn cao.** Bài toán giám sát và điều khiển một căn phòng là nhu cầu có thật, dễ kiểm chứng kết quả: nhìn đèn sáng trên mạch, nhìn số đo đổi khi hà hơi vào cảm biến.
- **Bao phủ trọn chuỗi IoT.** Một sản phẩm nhỏ nhưng đi qua đủ các tầng của một hệ thống IoT thực thụ: cảm biến và vi điều khiển (thiết bị biên), giao thức truyền thông MQTT (tầng mạng), máy chủ xử lý và lưu trữ (tầng ứng dụng), giao diện web thời gian thực (tầng người dùng).
- **Bám sát nội dung học phần.** Đề tài vận dụng trực tiếp các kiến thức được giảng: mô hình publish/subscribe, mức QoS của MQTT, thiết kế cơ sở dữ liệu cho dữ liệu chuỗi thời gian, thiết kế API và kiểm thử.
- **Chi phí thấp, dễ triển khai.** Toàn bộ linh kiện có sẵn trên thị trường; phần mềm đều là mã nguồn mở, chạy được trên một máy tính cá nhân.
- **Có nhiều bài toán kỹ thuật đáng giải quyết.** Đồng bộ lệnh điều khiển với phản hồi của phần cứng, phát hiện thiết bị mất kết nối, xử lý số đo sai, đẩy dữ liệu tới trình duyệt theo thời gian thực đều là những vấn đề mà một hệ thống IoT thật cũng phải đối mặt.

1.3 Mục tiêu

Bảng 1. Mục tiêu của đề tài

#	Mục tiêu	Kết quả mong đợi
1	Thu thập số liệu môi trường tự động	ESP8266 đọc DHT11 và quang trở, gửi về máy chủ mỗi 2 giây qua MQTT
2	Hiển thị số liệu theo thời gian thực	Dashboard cập nhật thẻ số liệu và biểu đồ không cần tải lại trang
3	Điều khiển thiết bị từ xa	Bật/tắt đèn và quạt từ trình duyệt, có xác nhận từ phần cứng
4	Lưu trữ và tra cứu lịch sử	Toàn bộ số liệu và thao tác được lưu vào CSDL, tra cứu có lọc, sắp xếp và phân trang

#	Mục tiêu	Kết quả mong đợi
5	Bảo vệ truy cập	Người dùng phải đăng nhập mới xem và điều khiển được; mỗi lệnh ghi lại người thao tác
6	Xử lý được tình huống lỗi	Thiết bị mất kết nối, số đo sai ngưỡng, broker hỏng đều có cơ chế xử lý rõ ràng

1.4 Phạm vi đề tài

Trong phạm vi: một căn phòng, hai thiết bị chấp hành (đèn và quạt, mô phỏng bằng LED), hai cảm biến đo ba đại lượng (DHT11 cho nhiệt độ và độ ẩm, quang trở cho ánh sáng), đăng nhập bằng tài khoản, chạy trong mạng LAN nội bộ.

Ngoài phạm vi: quản lý nhiều phòng, phân quyền theo vai trò, ứng dụng di động, cảnh báo qua email/SMS, điều khiển thiết bị điện lưới 220 V.

1.5 Yêu cầu chức năng

Bảng 2. Yêu cầu chức năng

Mã	Yêu cầu
FR-01	Tiếp nhận số liệu nhiệt độ, độ ẩm, ánh sáng từ thiết bị IoT qua topic data_sensors
FR-02	Hiển thị 3 thẻ số liệu tức thời kèm đơn vị (°C, %, lux)
FR-03	Hiển thị biểu đồ đường diễn biến số liệu theo thời gian
FR-04	Bật/tắt từng thiết bị độc lập từ giao diện
FR-05	Gửi lệnh điều khiển lên topic device_control
FR-06	Cập nhật trạng thái thiết bị theo phản hồi từ topic device_respond
FR-07	Lưu mọi số đo hợp lệ vào CSDL kèm mốc thời gian
FR-08	Lưu mọi thao tác điều khiển vào lịch sử kèm trạng thái thực thi
FR-09	Đánh dấu FAILED nếu không nhận phản hồi trong 5 giây
FR-10	Tìm kiếm và lọc dữ liệu theo cảm biến, giá trị, thời gian
FR-11	Sắp xếp theo cột và phân trang
FR-12	Cập nhật giao diện theo thời gian thực qua WebSocket
FR-13	Trang Profile chứa thông tin sinh viên và liên kết GitHub, báo cáo PDF, Figma, tài liệu API
FR-14	Loại bỏ số đo nằm ngoài ngưỡng hợp lệ
FR-15	Hiển thị trạng thái kết nối của thiết bị IoT (trực tuyến / ngoại tuyến)
FR-16	Quản lý danh mục cảm biến: mã, tên, đại lượng, đơn vị, ngưỡng hợp lệ riêng
FR-17	Ghi nhận người thao tác cho mỗi lệnh điều khiển
FR-18	Đăng nhập / đăng xuất; chưa đăng nhập thì không truy cập được các màn hình chức năng

1.6 Yêu cầu phi chức năng

Bảng 3. Yêu cầu phi chức năng

Mã	Loại	Yêu cầu
NFR-01	Hiệu năng	Từ lúc bấm nút tới lúc thiết bị đổi trạng thái ≤ 2 giây
NFR-02	Hiệu năng	Từ lúc thiết bị gửi số liệu tới lúc giao diện cập nhật ≤ 1 giây
NFR-03	Hiệu năng	API trả kết quả ≤ 500 ms với bảng tới 100.000 bản ghi
NFR-04	Độ tin cậy	Thiết bị tự kết nối lại WiFi và broker khi mất kết nối
NFR-05	Độ tin cậy	Không mất bản ghi lịch sử kể cả khi lệnh thất bại
NFR-06	Bảo mật	Broker bắt buộc xác thực username/password, tắt truy cập ẩn danh
NFR-07	Bảo mật	Kiểm tra hợp lệ toàn bộ dữ liệu đầu vào từ MQTT và HTTP
NFR-08	Bảo trì	Cấu hình và thông tin bí mật tách khỏi mã nguồn (.env, secrets.h)
NFR-09	Mở rộng	Thêm cảm biến hoặc thiết bị chỉ cần thêm dữ liệu, không đổi cấu trúc bảng
NFR-10	Bảo trì	Mọi thay đổi cấu trúc CSDL thực hiện qua migration

1.7 Bố cục báo cáo

- **Chương 1** trình bày lý do chọn đề tài, mục tiêu, phạm vi và yêu cầu của hệ thống.
- **Chương 2** trình bày kiến trúc hệ thống (phần cứng, phần mềm, truyền thông) và kiến trúc tổng quan use case.
- **Chương 3** trình bày thiết kế chi tiết: công nghệ sử dụng, cơ sở dữ liệu, backend, frontend, API và firmware.
- **Chương 4** trình bày kết quả thực nghiệm trên phần cứng và phần mềm.
- **Chương 5** tổng kết, nêu hạn chế và hướng phát triển.

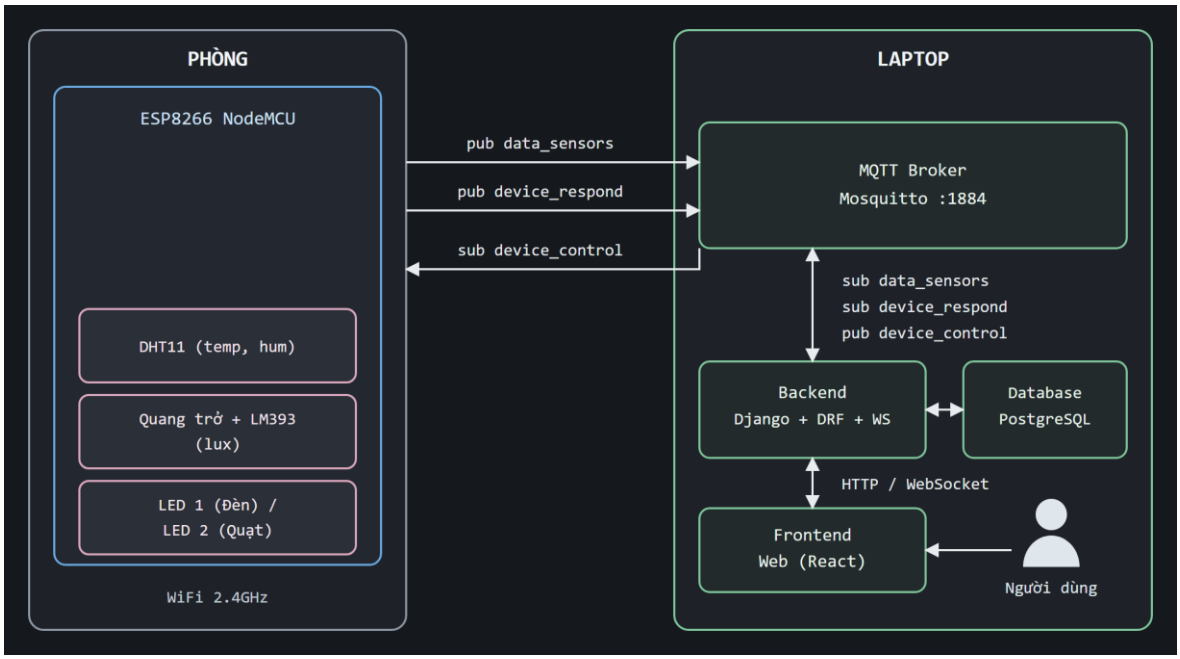
CHƯƠNG 2. KIẾN TRÚC HỆ THỐNG

2.1 Kiến trúc tổng thể

Hệ thống gồm hai khối đặt trong cùng một mạng WiFi nội bộ:

- **Khối thiết bị (Edge)**: mạch ESP8266 gắn cảm biến và LED, đặt trong phòng cần giám sát.
- **Khối máy chủ**: chạy trên một máy tính, gồm MQTT Broker (Mosquitto), Backend (Django), cơ sở dữ liệu (PostgreSQL), Redis và Frontend (React).

Hai khối không giao tiếp trực tiếp với nhau mà thông qua MQTT Broker theo mô hình publish/subscribe. Nhờ vậy thiết bị và máy chủ không cần biết địa chỉ của nhau và có thể khởi động lại độc lập. Trình duyệt giao tiếp với backend bằng hai kênh: HTTP REST cho truy vấn và điều khiển, WebSocket cho dữ liệu thời gian thực.



Hình 1. Kiến trúc tổng thể hệ thống

2.2 Kiến trúc phần cứng

2.2.1 Danh sách linh kiện

Bảng 4. Danh sách linh kiện

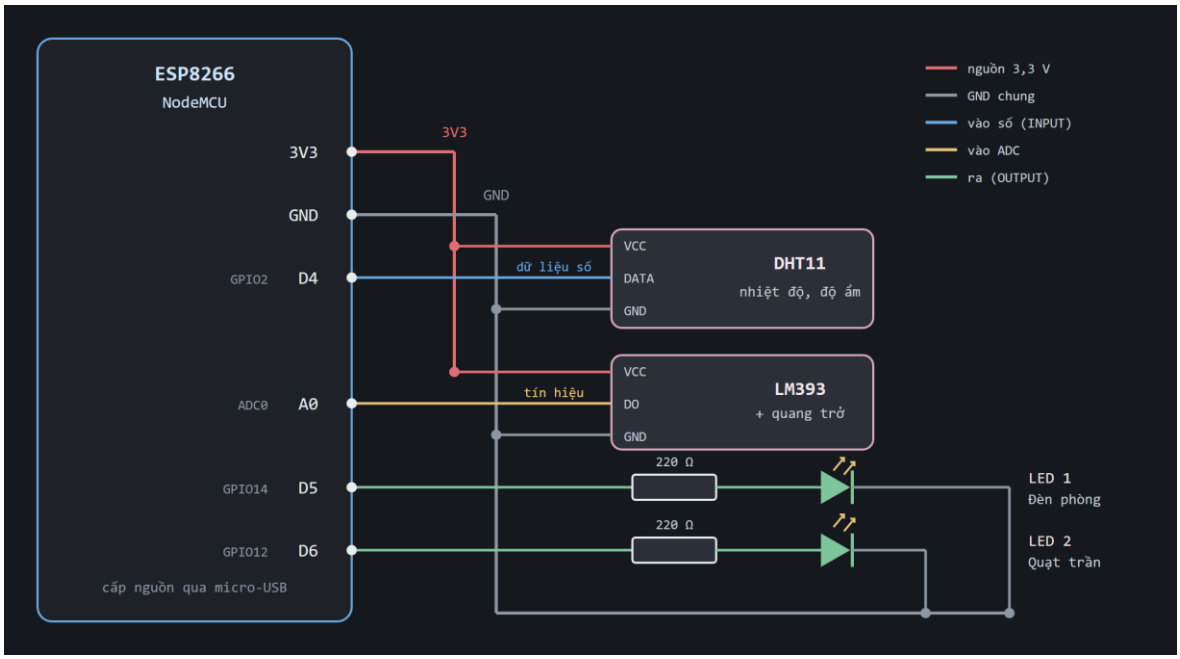
#	Linh kiện	Model	Chức năng
1	Vi điều khiển	ESP8266 NodeMCU	Đọc cảm biến, kết nối WiFi, điều khiển LED
2	Cảm biến nhiệt độ, độ ẩm	DHT11	Đo nhiệt độ (±2 °C) và độ ẩm (±5 %RH)
3	Cảm biến ánh sáng	Quang trở + module LM393	Đo cường độ chiếu sáng
4	Thiết bị chấp hành 1	LED + điện trở 220 Ω	Mô phỏng đèn phòng
5	Thiết bị chấp hành 2	LED + điện trở 220 Ω	Mô phỏng quạt trần

#	Linh kiện	Model	Chức năng
6	Phụ kiện	Breadboard, dây nối, cáp micro-USB	Lắp mạch và cấp nguồn

2.2.2 Sơ đồ đấu nối

Bảng 5. Đấu nối chân ESP8266

Chân ESP8266	Kết nối tới	Chế độ
D4 (GPIO2)	Chân Data của DHT11	INPUT
A0	Chân tín hiệu của module LM393	ANALOG INPUT
D5 (GPIO14)	LED 1 (Đèn phòng) qua trở 220 Ω	OUTPUT
D6 (GPIO12)	LED 2 (Quạt trần) qua trở 220 Ω	OUTPUT
3V3	VCC của DHT11 và LM393	—
GND	GND chung của toàn mạch	—



Hình 2. Sơ đồ đấu nối phần cứng

2.3 Kiến trúc phần mềm

2.3.1 Phân rã tiến trình phía Backend

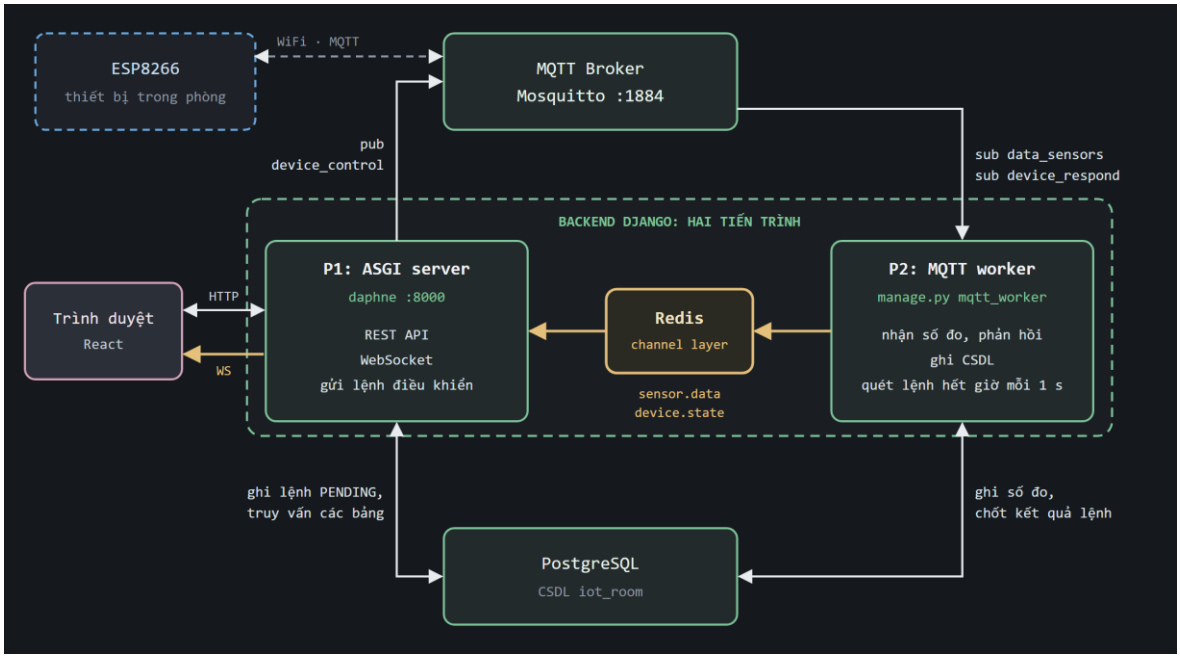
Django là framework đồng bộ theo mô hình request–response, không có sẵn vòng lặp nền để duy trì kết nối MQTT. Vì vậy backend được tách thành hai tiến trình chạy song song:

Bảng 6. Hai tiến trình phía Backend

Tiến trình	Lệnh khởi chạy	Trách nhiệm
P1: ASGI server	daphne -p 8000 config.asgi:application	Phục vụ REST API và WebSocket; gửi lệnh lên device_control

Tiến trình	Lệnh khởi chạy	Trách nhiệm
P2: MQTT worker	python manage.py mqtt_worker	Nhận data_sensors, device_respond; ghi CSDL; quét lệnh hết giờ mỗi giây; đẩy sự kiện sang P1

Redis là thành phần bắt buộc. Hai tiến trình nằm ở hai vùng nhớ khác nhau nên P2 không thể gọi trực tiếp tới các kết nối WebSocket do P1 đang giữ. Redis đóng vai trò channel layer: P2 đẩy sự kiện vào Redis, P1 nhận và phát tiếp tới trình duyệt.



Hình 3. Phân rã tiến trình phía Backend

2.3.2 Giao thức truyền thông MQTT

Hệ thống dùng đúng ba topic. Broker Mosquitto chạy ở cổng 1884, bắt buộc xác thực bằng tài khoản.

Bảng 7. Ba topic MQTT của hệ thống

Topic	Hướng	QoS	Nội dung
data_sensors	ESP8266 → Backend	0	Số liệu cảm biến, gửi mỗi 2 giây
device_control	Backend → ESP8266	1	Lệnh bật/tắt thiết bị
device_respond	ESP8266 → Backend	1	Xác nhận đã thực thi lệnh; báo trạng thái khi vừa kết nối lại

data_sensors dùng QoS 0 vì mất một mẫu trong hàng chục nghìn mẫu mỗi ngày không ảnh hưởng tới biểu đồ. device_control và device_respond dùng QoS 1 vì mất một lệnh điều khiển là người dùng thấy hệ thống hỏng. Không topic nào dùng cờ retain, để ESP8266 khởi động lại không nhận lại lệnh cũ và tự bật đèn.

Cấu trúc dữ liệu của ba topic:

```
// data_sensors
{ "device_id": "esp8266_room01", "temperature": 28.5,
  "humidity": 72.0, "light": 350 }

// device_control
{ "request_id": "3f2b8c1e-9a4d-4f7b-8c21-0e5d6a7b8c90",
  "device": "room01_lamp", "action": "ON" }

// device_respond
{ "request_id": "3f2b8c1e-9a4d-4f7b-8c21-0e5d6a7b8c90",
  "device": "room01_lamp", "state": "ON", "status": "SUCCESS" }
```

Trường request_id là mã duy nhất của mỗi lệnh, do backend sinh bằng uuid4(). Lệnh đi ra và phản hồi đi về theo hai đường khác nhau, nên không có mã này thì backend không biết phản hồi vừa nhận ứng với lệnh nào.

2.3.3 Luồng dữ liệu chính

Luồng thu thập: cứ 2 giây ESP8266 đọc cảm biến và gửi một gói data_sensors. MQTT worker tách gói thành ba số đo, kiểm tra ngưỡng của từng cảm biến, lưu vào CSDL với cùng một mốc thời gian rồi đẩy sự kiện sensor.data qua Redis tới mọi trình duyệt đang mở Dashboard.

Luồng điều khiển: người dùng bấm công tắc, frontend gọi POST /api/devices/{id}/control. Backend ghi lịch sử với trạng thái PENDING, gửi lệnh lên device_control và trả ngay 202 Accepted. ESP8266 bật/tắt LED rồi trả lời trên device_respond; worker cập nhật lịch sử thành SUCCESS và đẩy sự kiện device.state để giao diện mở khoá công tắc. Quá 5 giây không có phản hồi thì lệnh bị đánh dấu FAILED.

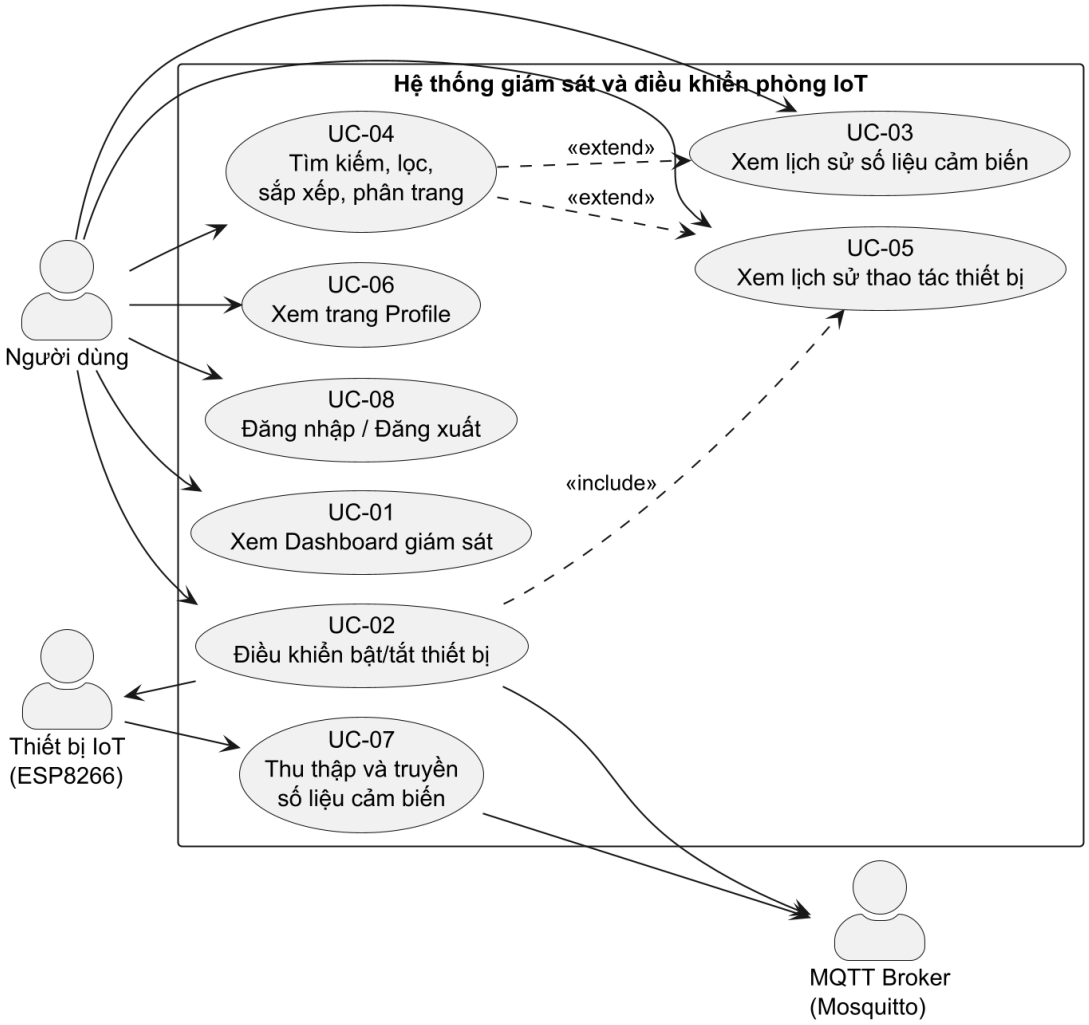
2.4 Kiến trúc tổng quan use case

2.4.1 Tác nhân

Bảng 8. Các tác nhân của hệ thống

Tác nhân	Mô tả
Người dùng	Người vận hành hệ thống qua trình duyệt: đăng nhập, xem số liệu, điều khiển thiết bị, tra cứu lịch sử
Thiết bị IoT	Mạch ESP8266, tự động gửi số liệu theo chu kỳ và phản hồi lệnh điều khiển
MQTT Broker	Tác nhân phụ, trung chuyển mọi thông điệp giữa thiết bị và máy chủ

2.4.2 Biểu đồ use case tổng quát



Hình 4. Biểu đồ use case tổng quát

Bảng 9. Danh sách use case

ID	Tên use case	Tác nhân chính	Mô tả ngắn
UC-01	Xem Dashboard giám sát	Người dùng	3 thẻ số liệu, biểu đồ 20 chu kỳ gần nhất, trạng thái thiết bị, cập nhật thời gian thực
UC-02	Điều khiển bật/tắt thiết bị	Người dùng	Bật/tắt đèn, quạt; công tắc khoá trong lúc chờ phần cứng xác nhận
UC-03	Xem lịch sử số liệu cảm biến	Người dùng	Bảng số đo: ID, mã cảm biến, tên, giá trị, đơn vị, thời gian
UC-04	Tìm kiếm, lọc, sắp xếp, phân trang	Người dùng	Áp dụng cho bảng của UC-03 và UC-05
UC-05	Xem lịch sử thao tác thiết bị	Người dùng	Bảng lệnh điều khiển: thiết bị, hành động, người thao tác, kết quả, độ trễ
UC-06	Xem trang Profile	Người dùng	Thông tin sinh viên và các liên kết bàn giao

ID	Tên use case	Tác nhân chính	Mô tả ngắn
UC-07	Thu thập và truyền số liệu	Thiết bị IoT	Tự động đọc cảm biến và gửi về máy chủ mỗi 2 giây
UC-08	Đăng nhập / Đăng xuất	Người dùng	Xác thực bằng tài khoản; điều kiện tiên quyết của UC-01 đến UC-06

2.4.3 Quan hệ giữa các use case

Bảng 10. Quan hệ giữa các use case

Quan hệ	Chiều	Giải thích
«extend»	UC-04 → UC-03	Tìm kiếm/lọc/sắp xếp là tuỳ chọn; UC-03 vẫn hoàn thành nếu người dùng không thao tác gì
«extend»	UC-04 → UC-05	Tương tự với bảng lịch sử thao tác
«include»	UC-02 → UC-05	Mỗi lần điều khiển luôn sinh một bản ghi lịch sử

2.4.4 Quy tắc nghiệp vụ

Bảng 11. Quy tắc nghiệp vụ

Mã	Quy tắc
BR-01	Thiết bị gửi số liệu theo chu kỳ 2 giây
BR-02	Mỗi cảm biến có ngưỡng hợp lệ riêng (nhiệt độ từ −10 đến 60 °C, độ ẩm từ 0 đến 100 %, ánh sáng từ 0 đến 2000 lux). Số đo ngoài ngưỡng bị loại riêng, các số đo khác cùng chu kỳ vẫn được lưu
BR-03	Quá 5 giây không nhận được phản hồi thì lệnh bị coi là thất bại
BR-04	Trong lúc một lệnh đang chờ, thiết bị đó không nhận lệnh mới: giao diện khoá công tắc và máy chủ từ chối bằng 409 DEVICE_BUSY
BR-05	Mỗi lệnh điều khiển sinh đúng một bản ghi lịch sử
BR-06	request_id là duy nhất trong toàn hệ thống
BR-07	Thiết bị bị coi là ngoại tuyến nếu không có số liệu mới trong 30 giây
BR-08	Phân trang mặc định 10 bản ghi/trang, tối đa 100
BR-09	Thứ tự sắp xếp mặc định: thời gian giảm dần
BR-10	Biểu đồ hiển thị 20 chu kỳ gần nhất
BR-11	Mỗi số đo là một bản ghi riêng; các số đo cùng chu kỳ mang cùng mốc thời gian
BR-12	Mỗi lệnh ghi lại người thao tác; không xác định được thì hiển thị dấu “—”
BR-13	Mọi thao tác trên giao diện và API đều yêu cầu đăng nhập

CHƯƠNG 3. THIẾT KẾ CHI TIẾT VÀ CÔNG NGHỆ SỬ DỤNG

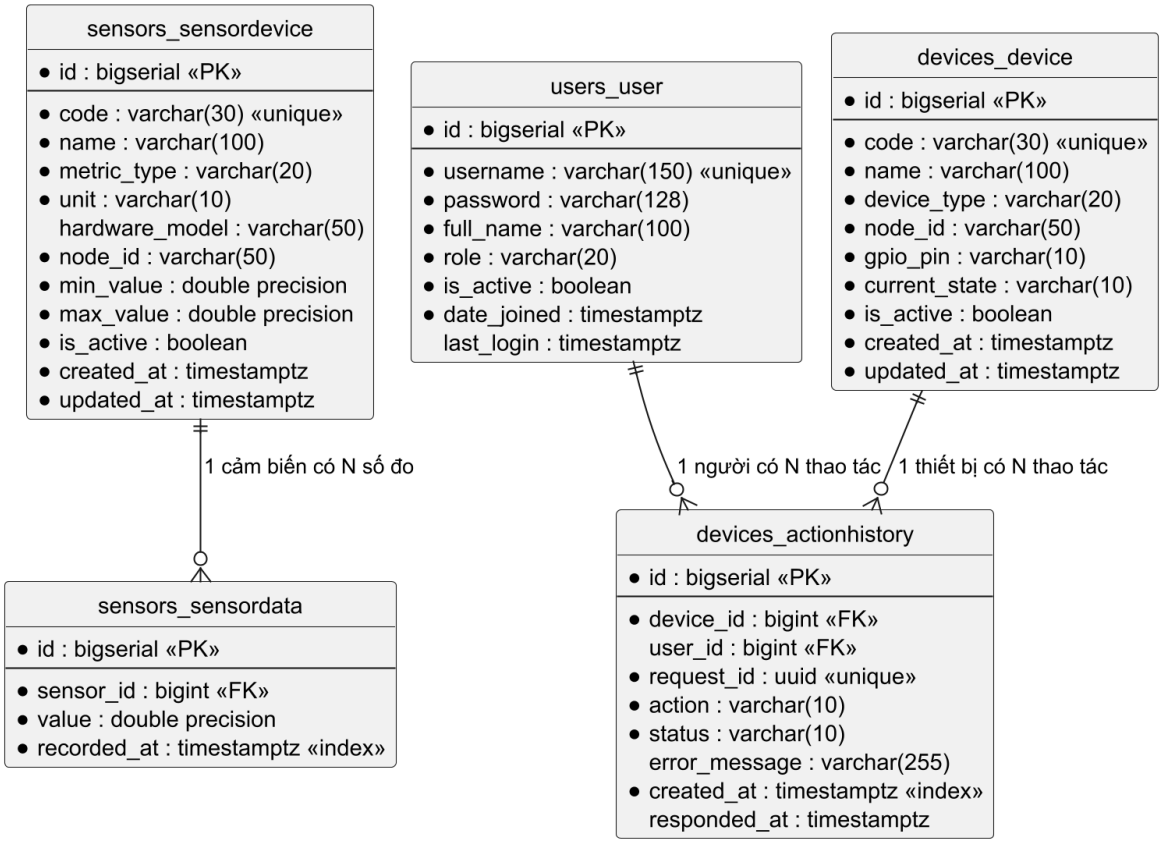
3.1 Công nghệ sử dụng

Bảng 12. Công nghệ sử dụng

Lớp	Công nghệ (phiên bản)	Lý do lựa chọn
Firmware	Arduino IDE (C++), ESP8266WiFi, DHT, arduino-mqtt (lwmqtt), ArduinoJson	arduino-mqtt publish được QoS 1; PubSubClient phổ biến hơn nhưng chỉ publish được QoS 0, không đáp ứng yêu cầu của topic device_respond
Broker	Eclipse Mosquitto 2.1, cổng 1884	Nhẹ, mã nguồn mở, chạy thành dịch vụ Windows; hỗ trợ xác thực tài khoản
Backend	Python 3.14, Django 5.2, Django REST Framework 3.18, django-filter	Có sẵn ORM, migration, trang quản trị, hệ thống lọc/phân trang/xác thực token
Realtime	Django Channels 4.3 + Daphne (ASGI)	Phục vụ HTTP và WebSocket trên cùng cổng 8000
Channel layer	Redis (container Docker) + channels-redis	Cầu nối giữa hai tiến trình backend
MQTT client	paho-mqtt 2.1	Thư viện MQTT chuẩn của Python
Cơ sở dữ liệu	PostgreSQL 18	Kiểu timestamptz, uuid, chỉ mục bộ phận, ràng buộc CHECK
Frontend	React 19, Vite 8, React Router 7, Recharts 3, Axios	Dựng giao diện nhiều trang và biểu đồ thời gian thực nhanh
Giả lập API	MSW (Mock Service Worker)	Làm giao diện song song khi backend chưa xong, không sửa code gọi API
Tài liệu API	drf-spectacular (OpenAPI 3, Swagger UI)	Sinh tài liệu tự động từ mã nguồn
Thiết kế UI	Figma	Thiết kế màn hình trước khi lập trình
Quản lý mã	Git, GitHub, GitHub Pages	Lưu lịch sử, công khai tài liệu API

3.2 Thiết kế cơ sở dữ liệu

Dữ liệu nghiệp vụ nằm trong năm bảng, chia làm ba nhóm: hai bảng danh mục mô tả cái gì đang tồn tại, hai bảng lịch sử ghi lại chuyện gì đã xảy ra, và một bảng người dùng cho biết ai đã làm. Ngoài ra Django tự tạo bảng authtoken_token lưu token đăng nhập của mỗi người dùng.



Hình 5. Sơ đồ thực thể liên kết (ERD)

Bảng 13. Các bảng dữ liệu nghiệp vụ

Bảng	**Vai trò**	**Nhịp ghi**
sensors_sensordevice	Danh mục cảm biến	Tĩnh, 3 dòng
sensors_sensordata	Số đo	Khoảng 129.600 dòng/ngày
devices_device	Thiết bị chấp hành	Tĩnh, 2 dòng
devices_actionhistory	Lịch sử thao tác	Vài chục dòng/ngày
users_user	Người dùng	Tĩnh

Quy ước đặt mã theo mẫu <vị trí>_<vai trò>: room01_temp, room01_humi, room01_lux, room01_lamp, room01_fan. Nhìn mã là biết thiết bị thuộc phòng nào; mở rộng sang phòng thứ hai chỉ cần thêm dữ liệu với tiền tố room02_.

3.2.1 Bảng danh mục cảm biến (sensors_sensordevice)

Bảng 14. Cấu trúc bảng danh mục cảm biến

Cột	**Kiểu**	**Mô tả**
id	bigserial	Khoá chính
code	varchar(30) UNIQUE	Mã nghiệp vụ: room01_temp, room01_humi, room01_lux
name	varchar(100)	Tên hiển thị, ví dụ “Nhiệt độ phòng”

Cột	Kiểu	Mô tả
metric_type	varchar(20)	TEMPERATURE / HUMIDITY / LIGHT
unit	varchar(10)	°C, %, lux
hardware_model	varchar(50)	Linh kiện thực tế: DHT11, LM393
node_id	varchar(50)	Bo mạch chứa cảm biến
min_value, max_value	double precision	Ngưỡng hợp lệ riêng của cảm biến (BR-02)
is_active	boolean	false khi cảm biến đã tháo
created_at, updated_at	timestamptz	Mốc tạo và lần sửa gần nhất

Đây là điểm mở rộng chính của thiết kế dữ liệu: lắp thêm một cảm biến (chẳng hạn cho phòng thứ hai) chỉ cần thêm một dòng vào bảng này, không phải thêm cột hay đổi cấu trúc bảng; MQTT worker nạp danh mục lúc khởi động nên cần khởi động lại để nhận cảm biến mới. Ràng buộc UNIQUE (node_id, metric_type) cho phép worker ánh xạ trường temperature trong gói MQTT sang đúng một cảm biến.

3.2.2 Bảng số đo (sensors_sensordata)

Bảng 15. Cấu trúc bảng số đo

Cột	Kiểu	Mô tả
id	bigserial	Khoá chính
sensor_id	bigint FK	Cảm biến sinh ra số đo
value	double precision NOT NULL	Giá trị đo; đơn vị lấy qua khoá ngoại
recorded_at	timestamptz	Thời điểm backend nhận số liệu

Mỗi bản ghi là một số đo của một cảm biến. Một chu kỳ 2 giây sinh ra ba bản ghi mang cùng recorded_at; ràng buộc UNIQUE (sensor_id, recorded_at) bảo đảm một cảm biến không có hai số đo tại cùng thời điểm. Cảm biến không đọc được thì không có bản ghi, nên cột value không bao giờ rỗng. Mốc thời gian do backend sinh vì ESP8266 không có đồng hồ thời gian thực.

3.2.3 Bảng thiết bị chấp hành (devices_device)

Bảng 16. Cấu trúc bảng thiết bị chấp hành

Cột	Kiểu	Mô tả
id	bigserial	Khoá chính, dùng trong URL API
code	varchar(30) UNIQUE	Mã nghiệp vụ dùng trong payload MQTT: room01_lamp, room01_fan
name	varchar(100)	Tên hiển thị
device_type	varchar(20)	LIGHT / FAN / OTHER
node_id, gpio_pin	varchar	Bo mạch và chân điều khiển; UNIQUE (node_id, gpio_pin)

Cột	Kiểu	Mô tả
current_state	varchar(10)	ON / OFF, chỉ đổi khi phần cứng xác nhận
is_active	boolean	false khi thiết bị đã tháo
created_at, updated_at	timestamptz	Mốc tạo và lần đổi trạng thái gần nhất

3.2.4 Bảng người dùng (users_user)

Bảng 17. Cấu trúc bảng người dùng

Cột	Kiểu	Mô tả
id	bigserial	Khoá chính
username	varchar(150) UNIQUE	Tên đăng nhập
password	varchar(128)	Mật khẩu đã băm (PBKDF2)
full_name	varchar(100)	Họ tên, hiển thị ở cột "Người thao tác"
role	varchar(20)	ADMIN / OPERATOR / VIEWER, hiện chỉ mang tính mô tả
is_active, date_joined, last_login	—	Các cột có sẵn của Django

Bảng kế thừa AbstractUser của Django: có sẵn băm mật khẩu, trang quản trị và tích hợp với cơ chế xác thực token của DRF.

3.2.5 Bảng lịch sử thao tác (devices_actionhistory)

Bảng 18. Cấu trúc bảng lịch sử thao tác

Cột	Kiểu	Mô tả
id	bigserial	Khoá chính
device_id	bigint FK	Thiết bị được điều khiển
user_id	bigint FK, cho phép NULL	Người thao tác (lệnh gửi bằng công cụ dòng lệnh thì rỗng)
request_id	uuid UNIQUE	Mã ghép cặp lệnh gửi đi với phản hồi nhận về
action	varchar(10)	ON / OFF
status	varchar(10)	PENDING / SUCCESS / FAILED
error_message	varchar(255)	Lý do thất bại
created_at	timestamptz	Thời điểm gửi lệnh, dùng làm mốc tính timeout 5 giây
responded_at	timestamptz	Thời điểm lệnh kết thúc; rỗng khi còn PENDING

Ràng buộc CHECK bảo đảm status = PENDING khi và chỉ khi responded_at rỗng. Cả ba khoá ngoại đặt on_delete=PROTECT: đối tượng ngừng dùng thì đặt is_active = false chứ không xoá, để bảo toàn lịch sử.

3.2.6 Chỉ mục

Bảng 19. Các chỉ mục

Bảng	**Cột**	**Phục vụ**
số đo	(recorded_at DESC, sensor_id)	Sắp xếp mặc định, lọc theo thời gian, phân trang ổn định
số đo	(sensor_id, recorded_at), sinh kèm ràng buộc UNIQUE	Số đo mới nhất từng cảm biến, biểu đồ, lọc theo cảm biến
lịch sử	created_at DESC	Sắp xếp mặc định màn Action History
lịch sử	(device_id, created_at DESC)	Lọc theo thiết bị
lịch sử	created_at WHERE status = 'PENDING'	Chỉ mục bộ phận cho vòng quét timeout chạy mỗi giây

3.3 Thiết kế Backend

3.3.1 Cấu trúc mã nguồn

Backend chia thành 6 ứng dụng Django. Mọi ứng dụng có cùng bộ file nên đọc tên file là biết tìm gì; logic nghiệp vụ nằm ở services.py, dùng chung cho cả API view lẫn MQTT worker.

Bảng 20. Các ứng dụng Django của Backend

Ứng dụng	**Trách nhiệm**
core	Thành phần dùng chung: sắp xếp có cột phá hoà, kiểm tra khoảng lọc, trang Profile
users	Bảng người dùng, đăng nhập / đăng xuất bằng token
sensors	Danh mục cảm biến, số đo, 4 endpoint /api/sensors*, ghi một chu kỳ đo
devices	Thiết bị, lịch sử thao tác, gửi lệnh, xác nhận, quét lệnh hết giờ
realtime	WebSocket: xác thực ?token=, consumer, 2 sự kiện
mqtt	Tiến trình 2: worker, xử lý message, ESP8266 giả lập để kiểm thử không cần mạch

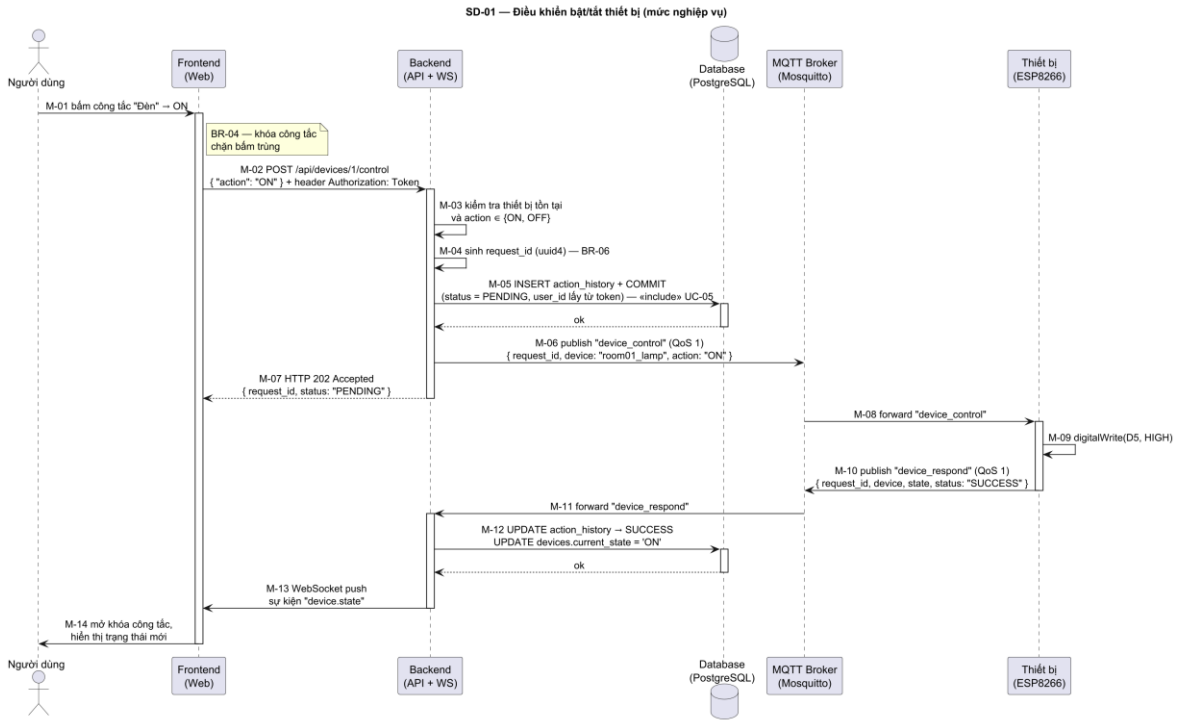
Bảng 21. Vai trò các file trong mỗi ứng dụng

File	**Chứa gì**
models.py	Bảng, ràng buộc, chỉ mục
serializers.py	Hình dạng JSON vào/ra
filters.py	Tham số lọc của bảng có phân trang
services.py	Logic nghiệp vụ
views.py	Chỉ đọc request, gọi service rồi trả kết quả
admin.py	Trang quản trị để xem/sửa dữ liệu

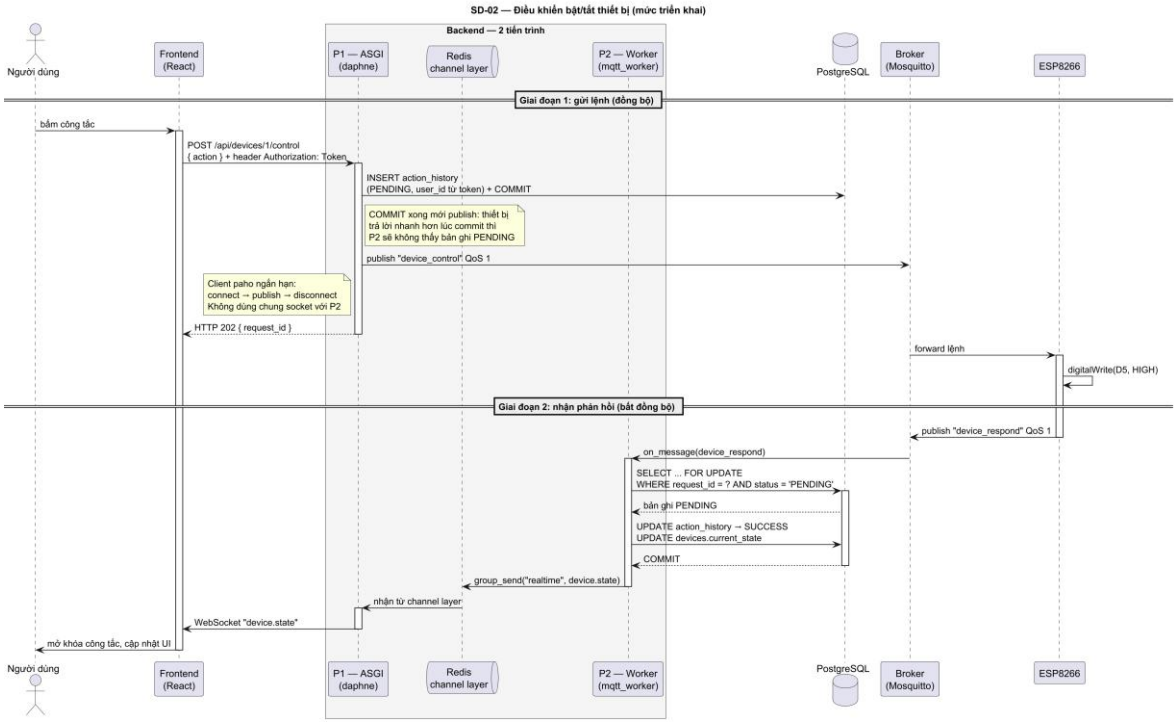
3.3.2 Xử lý lệnh điều khiển

Hàm send_command() thực hiện lần lượt: khoá dòng thiết bị (select_for_update) để hai tab gửi đồng thời không cùng tạo lệnh; nếu thiết bị còn lệnh PENDING thì trả 409 DEVICE_BUSY; tạo bản ghi PENDING kèm người thao tác lấy từ token; commit trước rồi mới publish lên broker. Nếu broker không nhận lệnh, bản ghi vừa tạo bị xoá và API trả 503 BROKER_UNAVAILABLE.

Thứ tự "commit rồi mới publish" là kết quả của một lỗi phát hiện khi chạy thật: nếu publish bên trong giao dịch, ESP8266 có thể trả lời (khoảng 20 ms) trước khi giao dịch kịp commit, worker không tìm thấy bản ghi PENDING và 5 giây sau đánh FAILED trong khi đèn đã bật.



Hình 6. Biểu đồ tuần tự điều khiển thiết bị (mức nghiệp vụ)

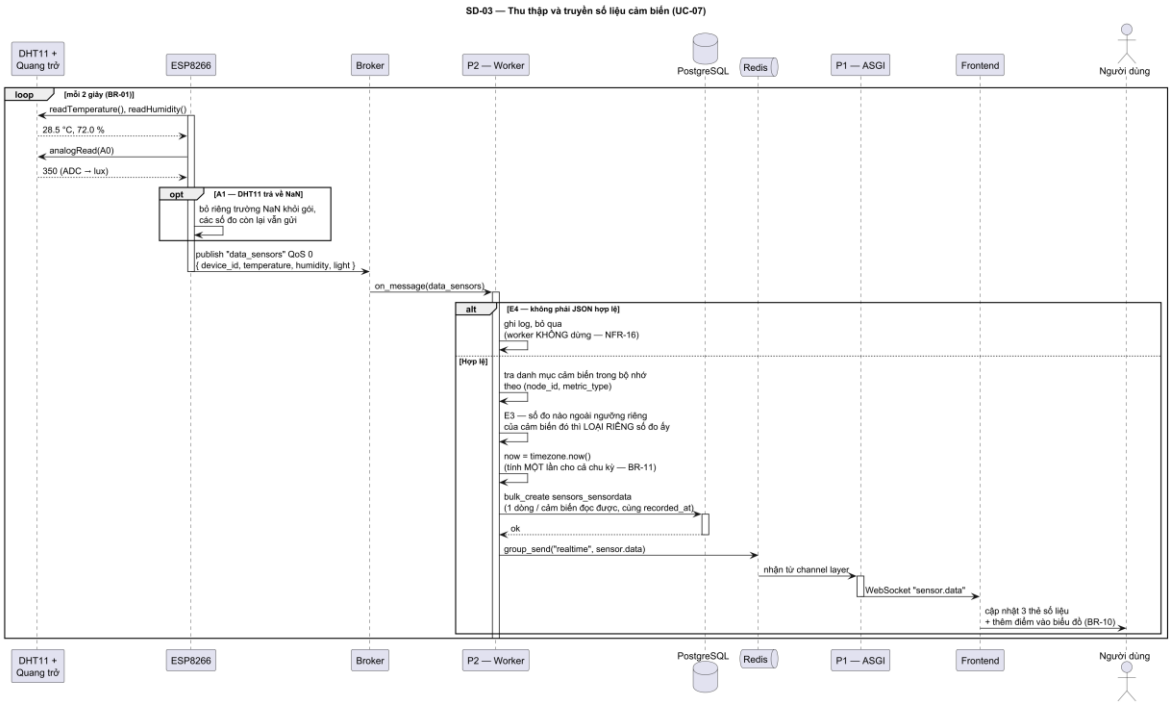


Hình 7. Biểu đồ tuần tự điều khiển thiết bị (mức triển khai)

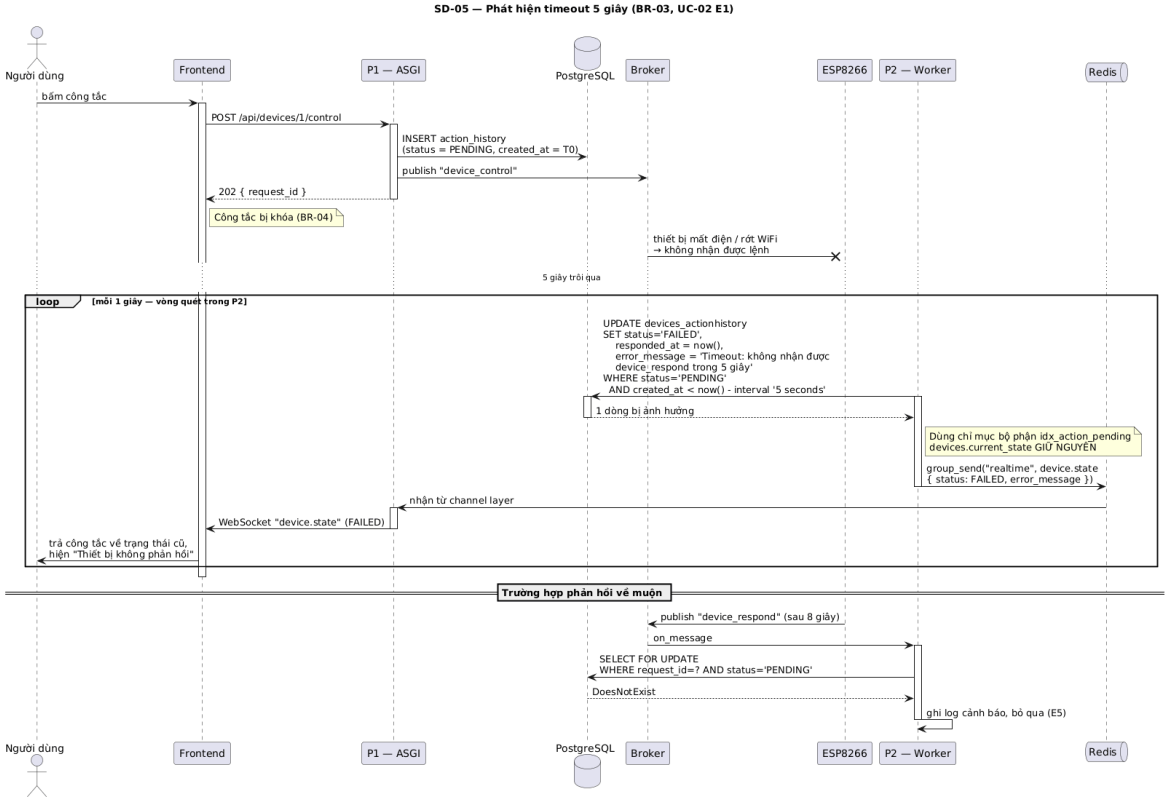
3.3.3 MQTT worker

Tiến trình mqtt_worker đảm nhận ba việc:

- **Nhận số đo** (record_cycle()): tách gói data_sensors thành ba số đo, đối chiếu ngưỡng trong danh mục, loại riêng số đo sai, ghi các số đo còn lại bằng một lệnh bulk_create với cùng một mốc thời gian, rồi phát sự kiện sensor.data.
- **Nhận phản hồi** (confirm_command()): tìm lệnh theo request_id, cập nhật SUCCESS cùng responded_at, đổi current_state của thiết bị và phát device.state. Gói có status = REPORT (thiết bị tự báo trạng thái khi vừa kết nối lại) chỉ đồng bộ current_state, không ghi lịch sử.
- **Quét lệnh hết giờ** (expire_pending_commands()): mỗi giây tìm lệnh PENDING quá 5 giây, đánh FAILED. Trạng thái nằm trong CSDL nên vẫn đúng kể cả khi worker khởi động lại; độ trễ thực tế của timeout là 5–6 giây.



Hình 8. Biểu đồ tuần tự thu thập và truyền số liệu cảm biến

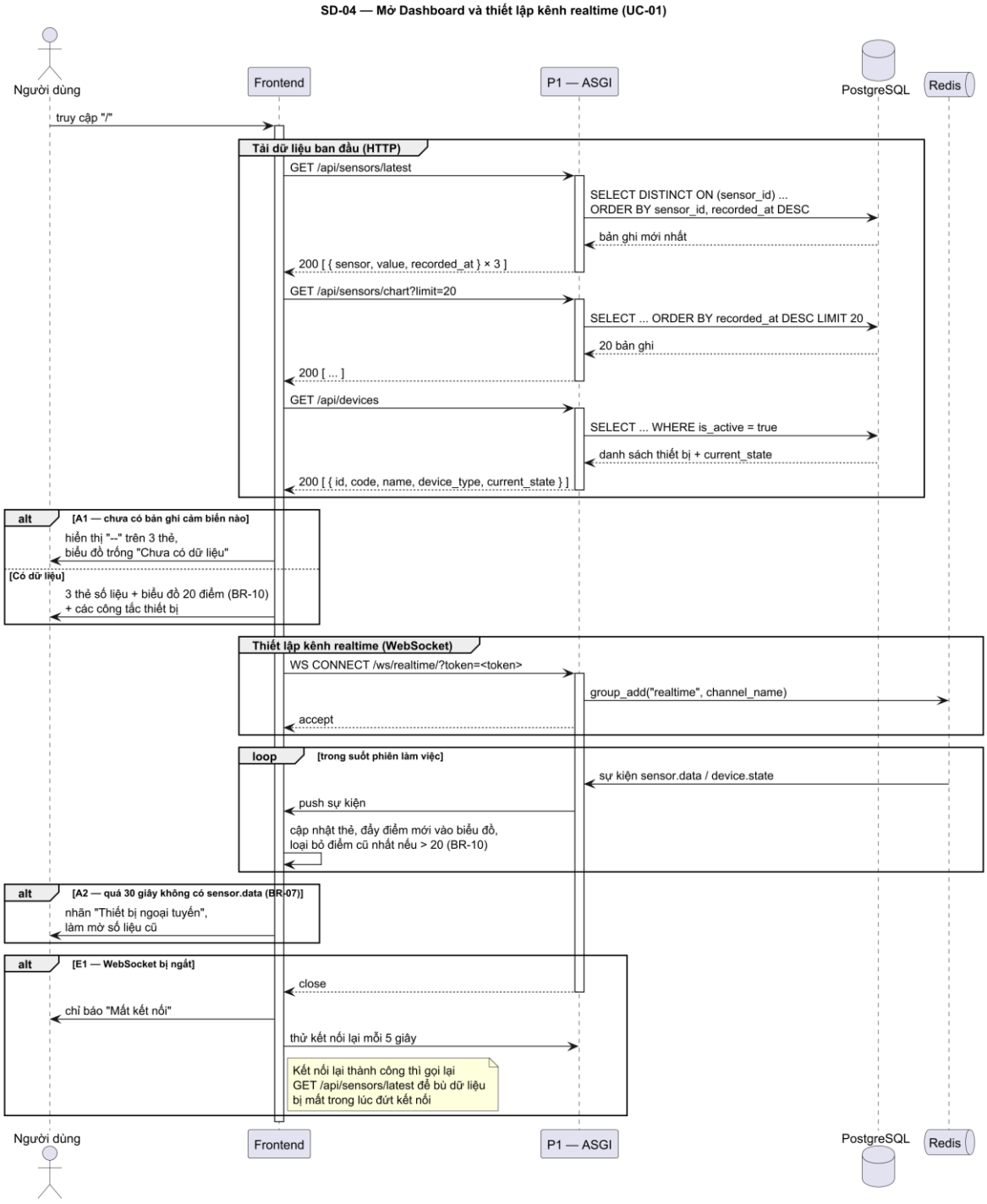


Hình 9. Biểu đồ tuần tự phát hiện lệnh quá thời gian chờ

3.3.4 Xác thực và thời gian thực

REST API dùng token của DRF: đăng nhập trả về một token, các request sau gửi kèm header Authorization: Token <key>. Thiếu hoặc sai token trả 401 UNAUTHENTICATED. WebSocket nhận token qua tham số ?token=; token sai thì máy chủ chấp nhận kết nối rồi đóng ngay với mã 4401 để frontend phân biệt được với lỗi mạng và không cố kết nối lại.

Mọi trình duyệt đã đăng nhập tham gia nhóm realtime trên Redis. Worker phát sự kiện vào nhóm này, Daphne chuyển tiếp tới từng kết nối WebSocket.



Hình 10. Biểu đồ tuần tự mở Dashboard và thiết lập kênh thời gian thực

3.4 Thiết kế Frontend

3.4.1 Cấu trúc mã nguồn

Bảng 22. Cấu trúc mã nguồn Frontend

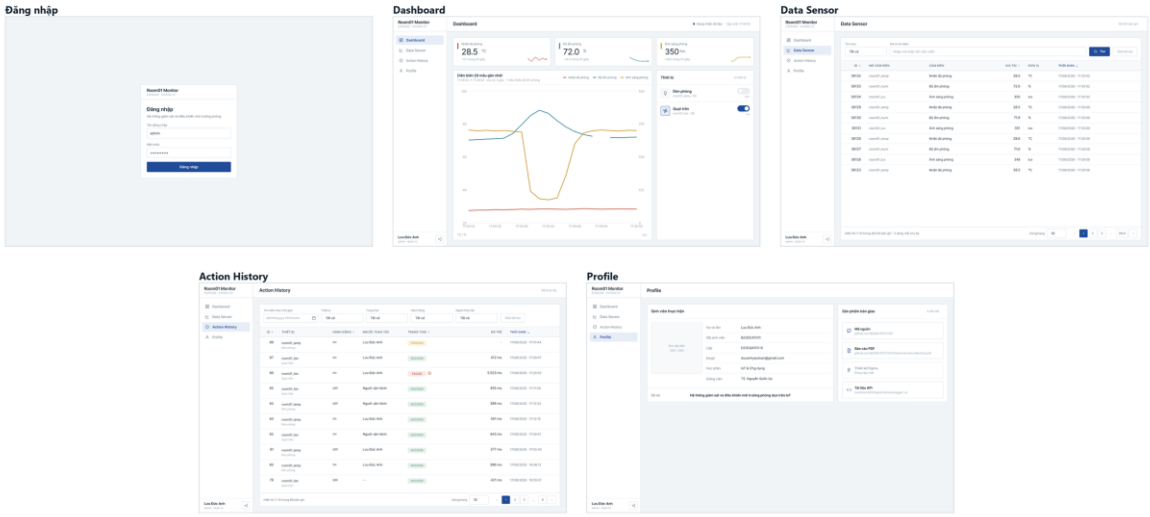
Thư mục	Nội dung
api/	client.js: một đối tượng Axios dùng chung, tự gắn token, chuẩn hoá lỗi; endpoints.js: danh sách mọi lời gọi API
auth/	AuthContext: giữ token, đăng nhập/đăng xuất, chặn trang khi chưa đăng nhập, đồng bộ giữa các tab
realtime/	RealtimeContext: một kết nối WebSocket cho cả ứng dụng, tự kết nối lại sau 5 giây
pages/	5 màn hình: Login, Dashboard, Data Sensor, Action History, Profile
components/	Layout (sidebar), bảng phân trang, tiêu đề cột sắp xếp, ô lọc, thẻ số liệu, biểu đồ, bảng điều khiển thiết bị
mocks/	MSW giả lập toàn bộ API và WebSocket bằng bộ dữ liệu mẫu
styles/	Bảng màu và thành phần dùng chung (CSS Modules)

Ứng dụng không dùng thư viện quản lý trạng thái: chỉ có hai thứ dùng chung toàn ứng dụng (thông tin đăng nhập và kết nối WebSocket), dữ liệu bảng của trang nào thì trang đó tự gọi API.

3.4.2 Các màn hình

Bảng 23. Chức năng các màn hình

Màn hình	Chức năng chính
Đăng nhập	Nhập tài khoản/mật khẩu; đăng nhập xong quay lại đúng trang đang xem trước đó
Dashboard	3 thẻ số liệu kèm biến thiên 20 giây; biểu đồ 20 chu kỳ (đường đứt khi thiếu số đo); công tắc đèn/quạt khoá "ĐANG GỬI…" khi chờ phản hồi; biểu tượng đèn sáng, quạt quay khi bật; nhãn "Thiết bị ngoại tuyến" sau 30 giây không có số liệu
Data Sensor	Bảng số đo; ô "Tìm theo" (tất cả / từng cảm biến / thời gian) + ô giá trị + nút Tìm; sắp xếp theo ID, giá trị, thời gian; chọn số dòng mỗi trang
Action History	Bảng lịch sử lệnh; tìm theo thời điểm (gõ gần đúng: cả ngày, trọn giờ, trọn phút hoặc đúng giây); lọc thiết bị, hành động, trạng thái, người thao tác
Profile	Thông tin sinh viên và 4 liên kết: GitHub, báo cáo PDF, Figma, tài liệu API



Hình 11. Thiết kế giao diện trên Figma

3.4.3 Các cơ chế xử lý chính

- **Khoá công tắc (BR-04):** bấm công tắc thì công tắc bị khoá, chỉ mở khoá khi nhận được sự kiện device.state có request_id khớp. Có bộ đếm dự phòng 7 giây để công tắc không bị khoá vĩnh viễn nếu WebSocket đứt đúng lúc.
- **Phát hiện ngoại tuyến (BR-07):** mỗi sự kiện sensor.data đặt lại bộ đếm 30 giây; hết giờ thì hiện nhãn ngoại tuyến. Backend không phải đếm giờ.
- **Múi giờ:** API luôn trả giờ UTC; frontend đổi sang giờ Việt Nam khi hiển thị và gửi bộ lọc thời gian kèm +07:00. Cận trên của khoảng lọc lấy tới .999999 giây vì CSDL lưu tới micro-giây.
- **Phiên hết hạn:** mọi phản hồi 401 đưa người dùng về trang đăng nhập.

3.5 Thiết kế API

3.5.1 Quy ước chung

Bảng 24. Quy ước chung của API

Mục	Quy ước
Địa chỉ gốc	http://<máy chủ>:8000/api
Định dạng	JSON, tên trường snake_case
Xác thực	Header Authorization: Token <key> cho mọi endpoint trừ đăng nhập
Thời gian	ISO 8601, múi giờ UTC (hậu tố Z)
Phân trang	?page= và ?page_size= (từ 1 đến 100); trả count, next, previous, results
Cấu trúc lỗi	{"error": {"code", "message", "details"}} thống nhất cho mọi lỗi
Tài liệu	Swagger UI sinh tự động tại /api/schema/swagger-ui/

3.5.2 Danh sách endpoint

Bảng 25. Danh sách endpoint

#	Phương thức	Đường dẫn	Chức năng
1	POST	/api/auth/login	Đăng nhập, trả token và thông tin người dùng
2	POST	/api/auth/logout	Đăng xuất, thu hồi token
3	GET	/api/sensors/devices	Danh mục cảm biến
4	GET	/api/sensors/latest	Số đo mới nhất của từng cảm biến
5	GET	/api/sensors/chart	20 chu kỳ gần nhất cho biểu đồ
6	GET	/api/sensors	Lịch sử số đo: tìm kiếm, lọc, sắp xếp, phân trang
7	GET	/api/devices	Danh sách thiết bị và trạng thái hiện tại
8	POST	/api/devices/{id}/control	Gửi lệnh bật/tắt, trả 202 kèm request_id
9	GET	/api/actions	Lịch sử thao tác: lọc, sắp xếp, phân trang
10	GET	/api/profile	Thông tin trang Profile
—	WS	/ws/realtime/?token=	Kênh thời gian thực: sự kiện sensor.data, device.state

3.5.3 Tham số lọc

Bảng 26. Tham số lọc của hai bảng lịch sử

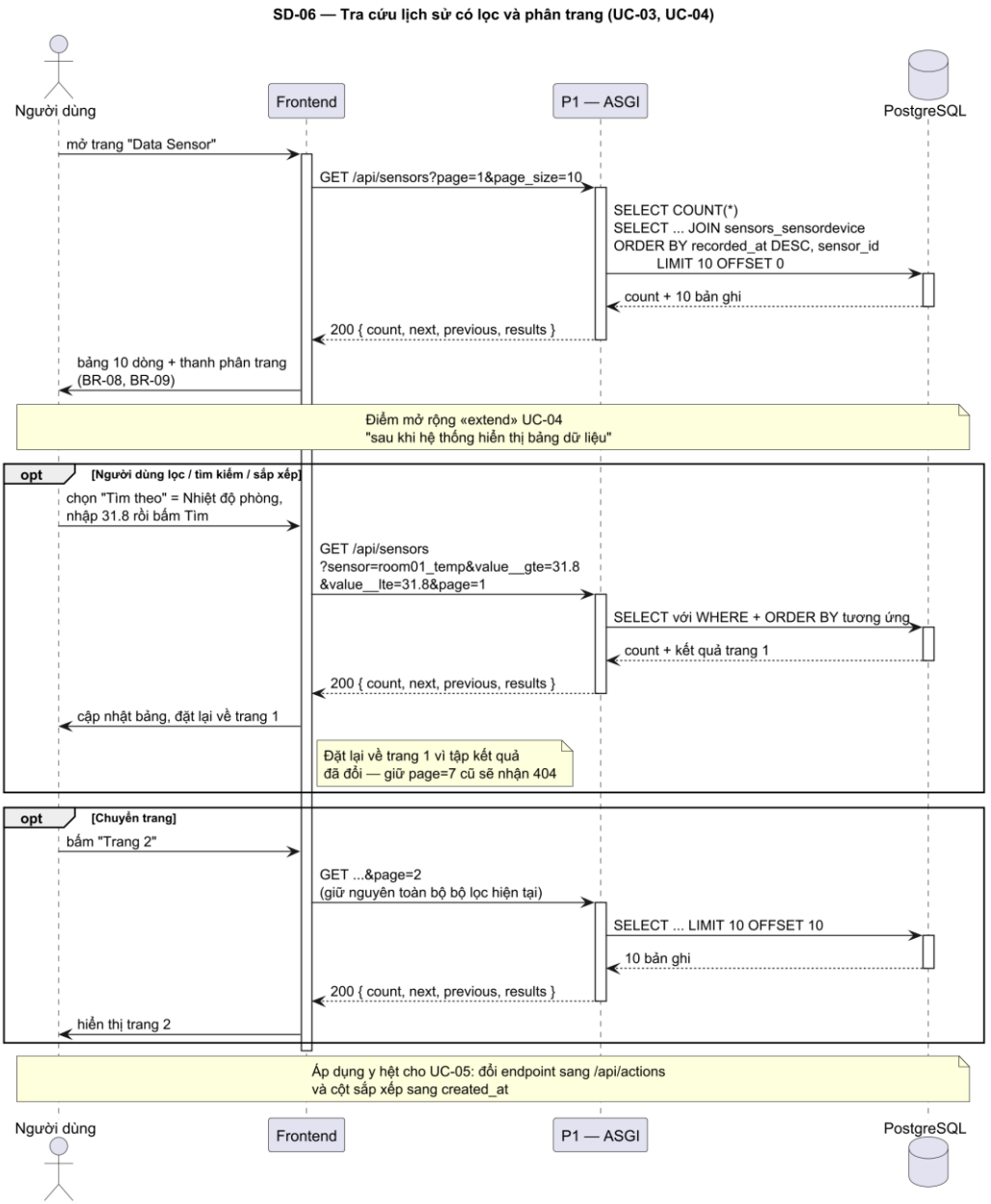
Endpoint	Tham số
/api/sensors	search (mã, tên cảm biến); sensor (mã cảm biến); value__gte, value__lte (bắt buộc kèm sensor); recorded_at__gte, recorded_at__lte; ordering (id, value, recorded_at)
/api/actions	search; device; action; status; user (id hoặc none); created_at__gte, created_at__lte; ordering

3.5.4 Mã lỗi

Bảng 27. Mã lỗi của API

HTTP	Mã	Ý nghĩa
400	VALIDATION_ERROR	Tham số hoặc dữ liệu gửi lên sai kiểu, sai giá trị
400	INVALID_RANGE	Đầu khoảng lớn hơn cuối khoảng (thời gian hoặc giá trị)
400	INVALID_CREDENTIALS	Sai tên đăng nhập hoặc mật khẩu
401	UNAUTHENTICATED	Chưa đăng nhập hoặc token đã thu hồi
404	NOT_FOUND	Không có thiết bị (hoặc thiết bị đã ngừng dùng)

HTTP	Mã	Ý nghĩa
404	PAGE_NOT_FOUND	Số trang vượt quá tổng số trang
409	DEVICE_BUSY	Thiết bị đang có lệnh chờ phản hồi
503	BROKER_UNAVAILABLE	Không gửi được lệnh tới MQTT Broker
500	INTERNAL_ERROR	Lỗi không lường trước của máy chủ



Hình 12. Biểu đồ tuần tự tra cứu lịch sử có lọc và phân trang

3.6 Thiết kế Firmware

Firmware viết bằng Arduino C++, các thông tin bí mật (WiFi, tài khoản MQTT) tách ra file secrets.h không đưa lên kho mã. Hoạt động chính:

- setup(): cấu hình chân LED, khởi động DHT11, đọc nháp và bỏ lần đọc đầu tiên (lần đọc đầu sau khi cấp điện trả số sai như 0,9 °C / 0 %), kết nối WiFi và broker.
- loop(): duy trì kết nối MQTT (tự kết nối lại khi mất), cứ 2 giây gọi publishSensorData() gửi gói data_sensors; giá trị NaN của DHT11 bị bỏ khỏi gói thay vì gửi số sai.
- Nhận device_control: so khớp mã thiết bị, bật/tắt chân GPIO tương ứng, trả device_respond kèm đúng request_id.
- publishStateReport(): mỗi lần kết nối hoặc kết nối lại tới broker, gửi trạng thái thực của từng thiết bị (status = REPORT) để giao diện khớp với mạch sau khi mạch mất điện.
- Firmware khai báo bộ đệm MQTT 256 byte (mặc định của arduino-mqtt chỉ 128 byte, sát kích thước gói device_control).

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM

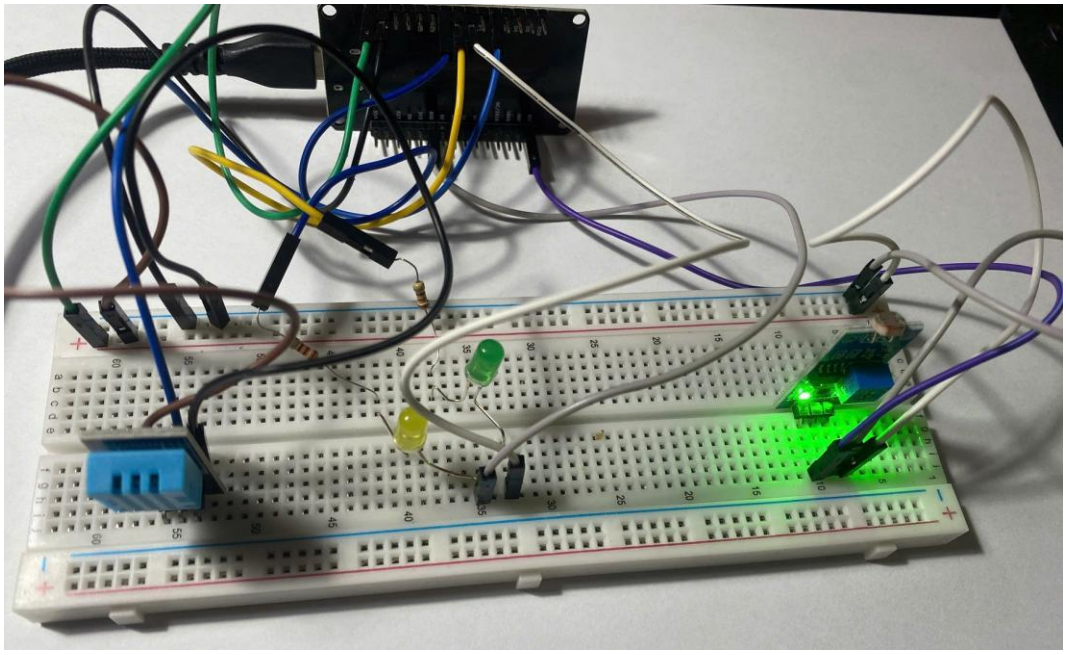
4.1 Môi trường triển khai

Bảng 28. Môi trường triển khai

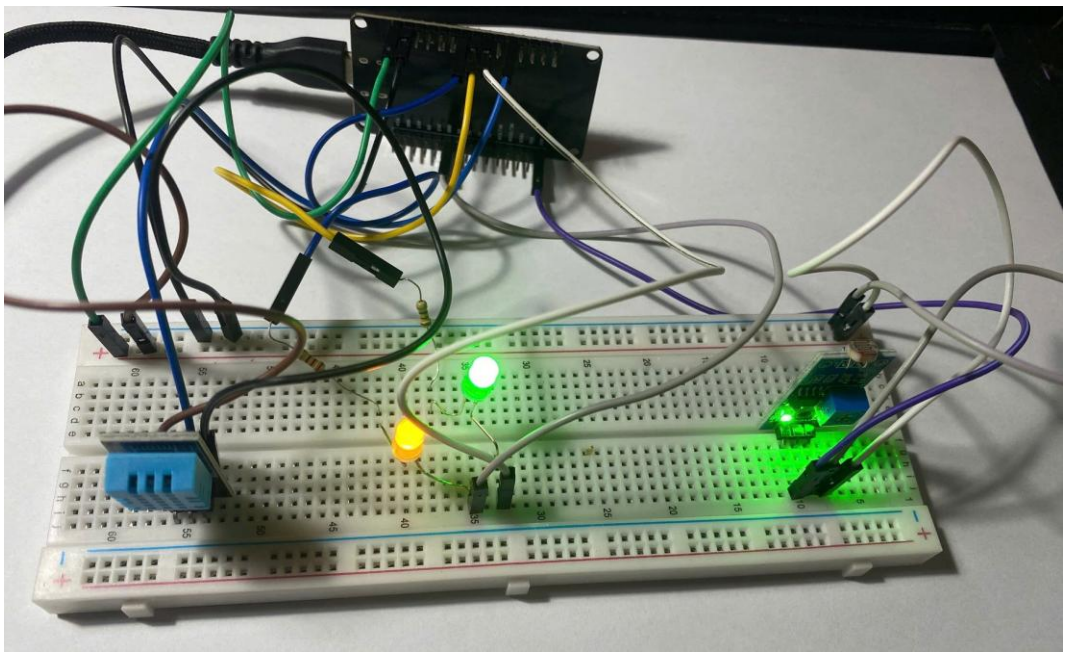
Thành phần	Cấu hình
Máy chủ	Laptop Windows 11 Pro
Backend	Python 3.14, Django 5.2, Daphne, cổng 8000
Cơ sở dữ liệu	PostgreSQL 18, CSDL iot_room
Redis	Container Docker redis-dev
MQTT Broker	Mosquitto 2.1, cổng 1884, bật xác thực
Frontend	Vite dev server, cổng 5173; trình duyệt Chrome
Thiết bị	ESP8266 NodeMCU + DHT11 + LM393 + 2 LED, kết nối cùng mạng WiFi với laptop

4.2 Kết quả phần cứng

Mạch được lắp trên breadboard theo sơ đồ đấu nối ở chương 2, nạp firmware qua Arduino IDE và kết nối thành công tới broker qua WiFi.



Hình 13. Mạch phần cứng sau khi lắp ráp



Hình 14. Đèn và quạt (LED) được bật từ giao diện web

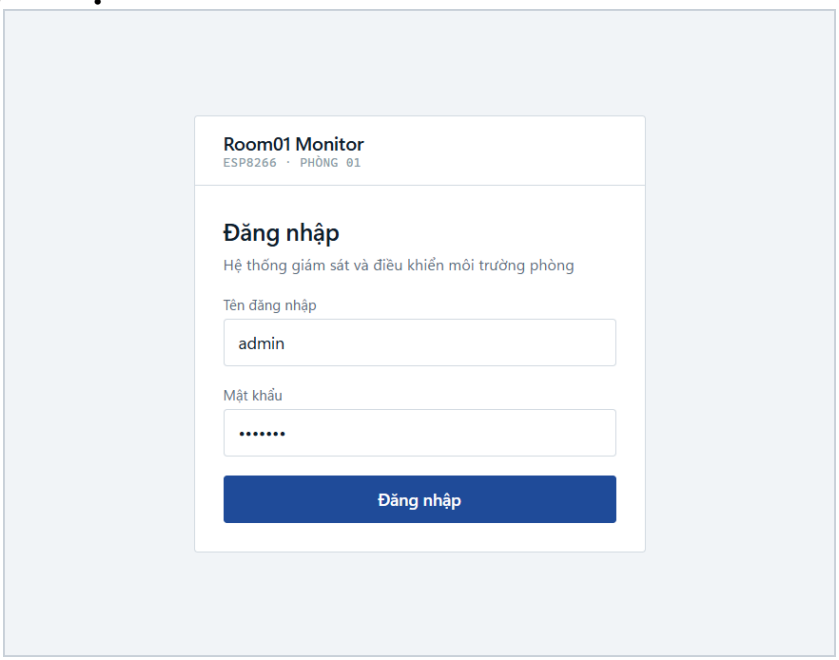
```
=== esp8266_room01 — firmware 2026-09-24 lenh-gop ===
Dang ket noi WiFi..... OK, IP: 172.20.10.2
Dang ket noi MQTT broker voi tai khoan "iotuser" (client ID: esp8266_room01_f6d0cb) OK
Da bao trang thai: {"device":"room01_lamp","state":"OFF","status":"REPORT"}
Da bao trang thai: {"device":"room01_fan","state":"OFF","status":"REPORT"}
San sang. Go lenh vao o Message cua Serial Monitor:
  lamp on | lamp off | fan on | fan off | all on | all off
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.1,"humidity":34,"light":40}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.4,"humidity":33,"light":40}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.3,"humidity":33,"light":40}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.8,"humidity":33,"light":40}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.8,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.9,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.7,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.7,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.6,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.5,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.6,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.1,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.1,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.7,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.7,"humidity":33,"light":46}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.9,"humidity":33,"light":46}
Nhan tren device_control: {"request_id": "fc7250b0-3424-41b2-a40d-237e7a456796", "device": "room01_lamp", "action": "ON"}
Da gui device_respond: {"request_id":"fc7250b0-3424-41b2-a40d-237e7a456796","device":"room01_lamp","state":"ON","status":"SUCCESS"}
Nhan tren device_control: {"request_id": "3c054611-b773-428e-a440-5ea66544a397", "device": "room01_lamp", "action": "OFF"}
Da gui device_respond: {"request_id":"3c054611-b773-428e-a440-5ea66544a397","device":"room01_lamp","state":"OFF","status":"SUCCESS"}
Da gui data_sensors: {"device_id":"esp8266_room01","temperature":31.9,"humidity":33,"light":46}
```

Hình 15. Nhật ký Serial Monitor của ESP8266

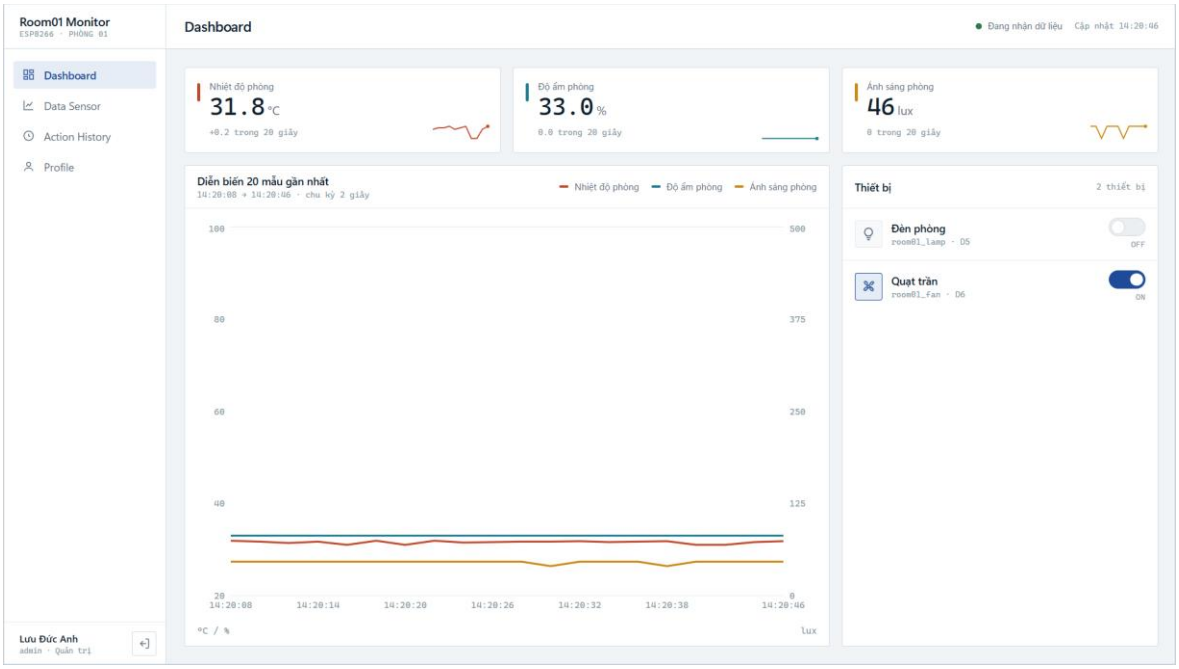
Kết quả chạy với mạch thật (18/09/2026):

- 110 chu kỳ đo đầu tiên đủ cả 3 số đo ở mọi chu kỳ; nhiệt độ, độ ẩm phản ánh đúng môi trường phòng: khoảng 30 °C và 78 % trong lần chạy này; lúc chụp Hình 15 (21/10/2026) phòng khô hơn nên độ ẩm chỉ khoảng 33 %.
- 16 lệnh bật/tắt đèn và quạt liên tiếp đều thành công, độ trễ khứ hồi từ lúc gửi lệnh tới lúc nhận xác nhận là 89–273 ms.
- Rút điện mạch khi đèn đang bật rồi cấp lại: mạch khởi động với đèn tắt và tự báo trạng thái, giao diện chuyển công tắc về OFF đúng với thực tế.

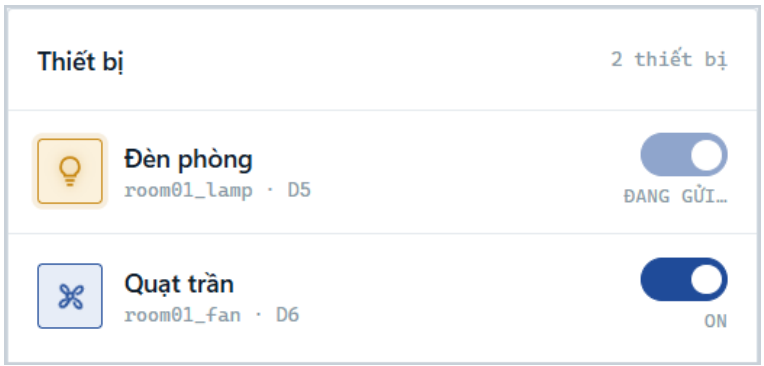
4.3 Kết quả giao diện



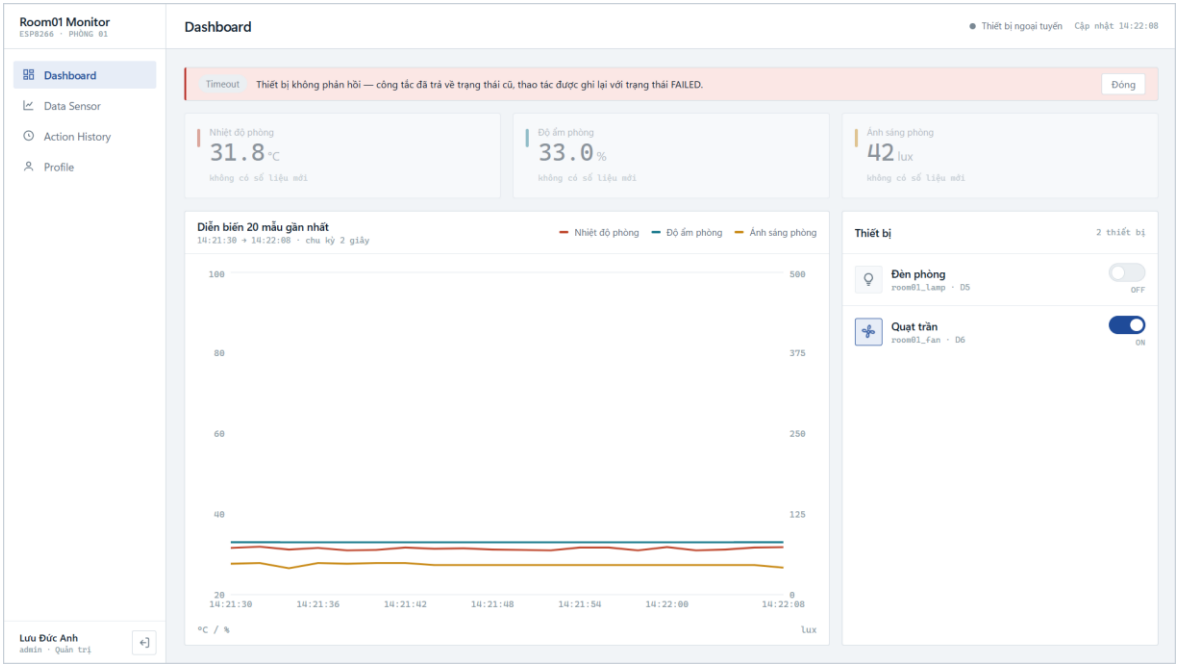
Hình 16. Màn hình đăng nhập



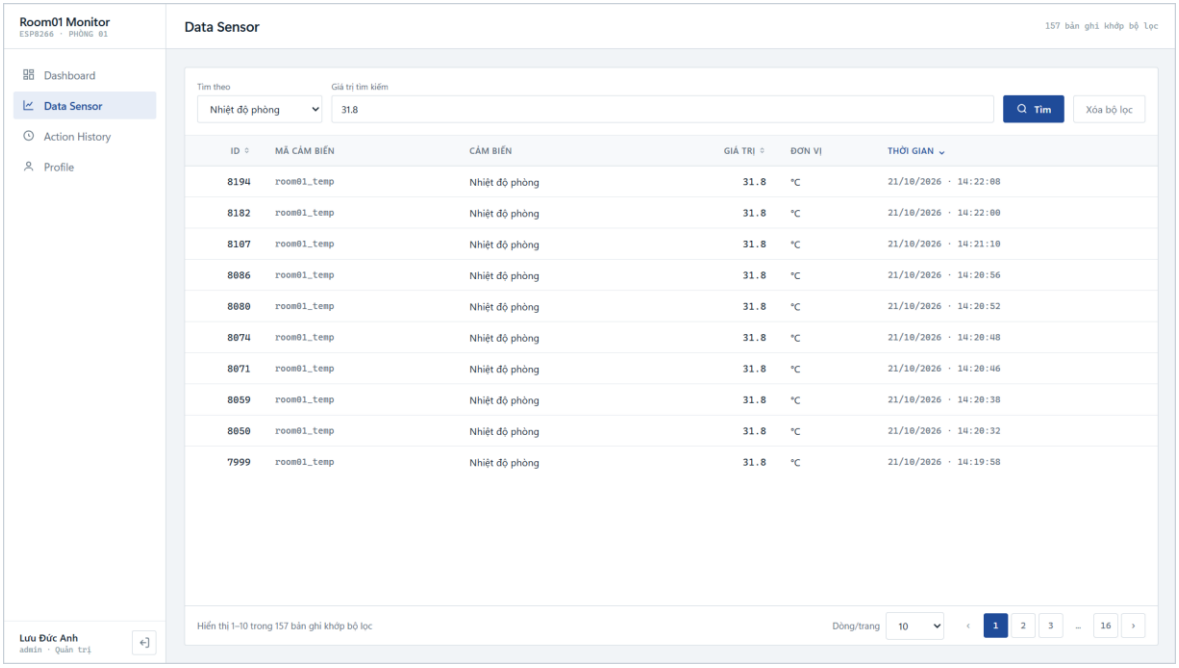
Hình 17. Màn hình Dashboard



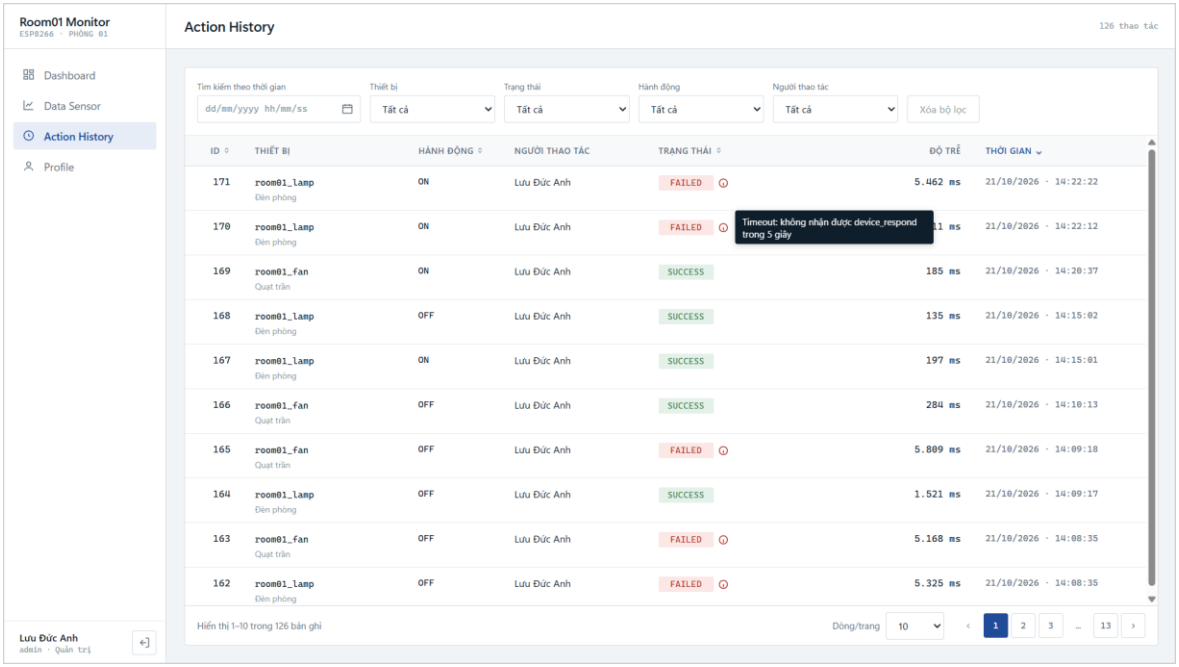
Hình 18. Công tắc đang chờ phản hồi từ thiết bị



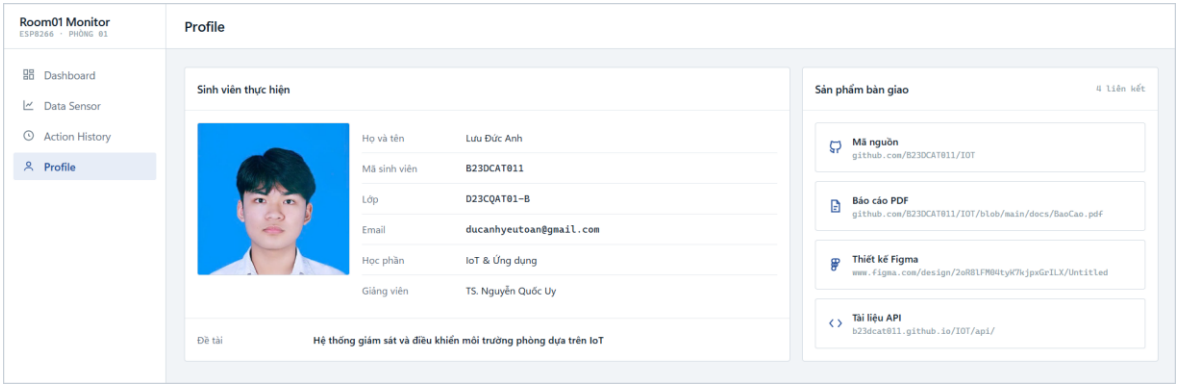
Hình 19. Dashboard khi thiết bị ngoại tuyến



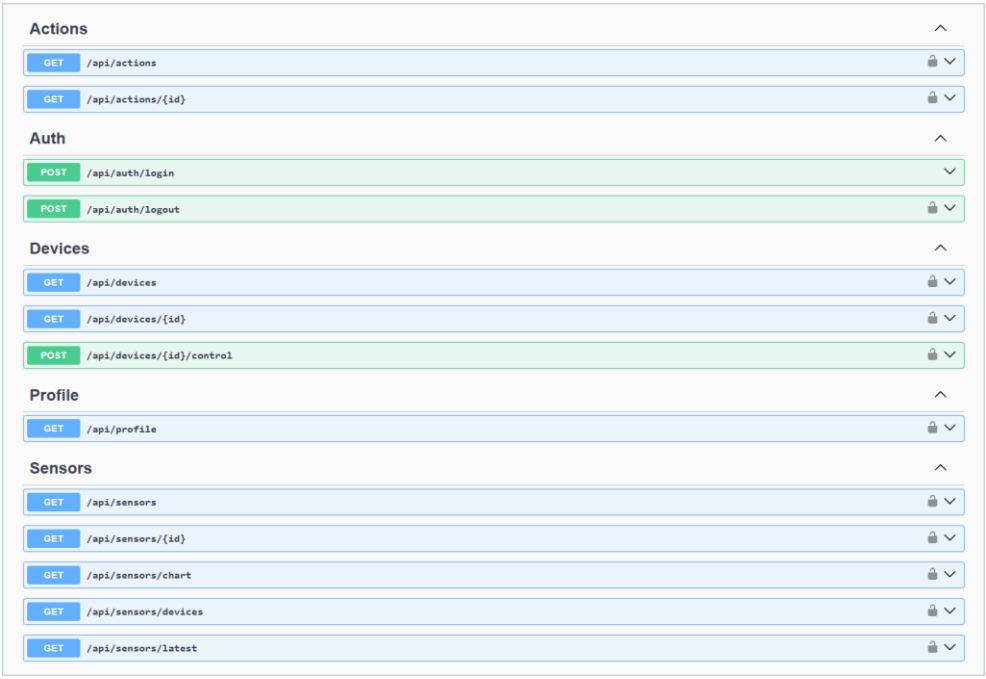
Hình 20. Màn hình Data Sensor



Hình 21. Màn hình Action History



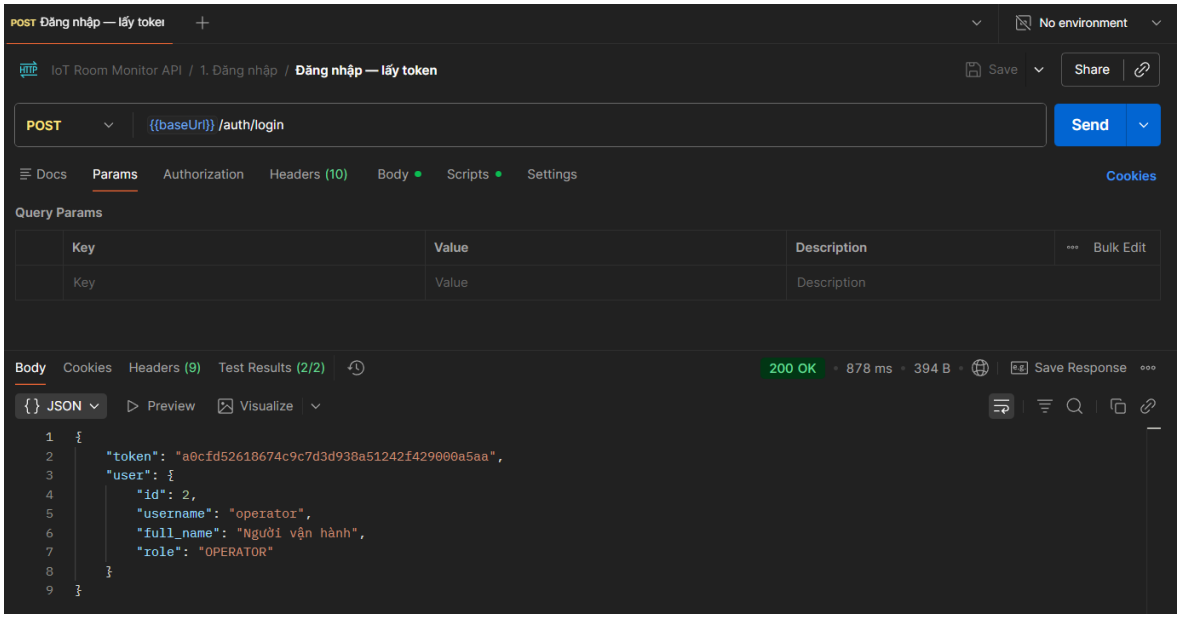
Hình 22. Màn hình Profile



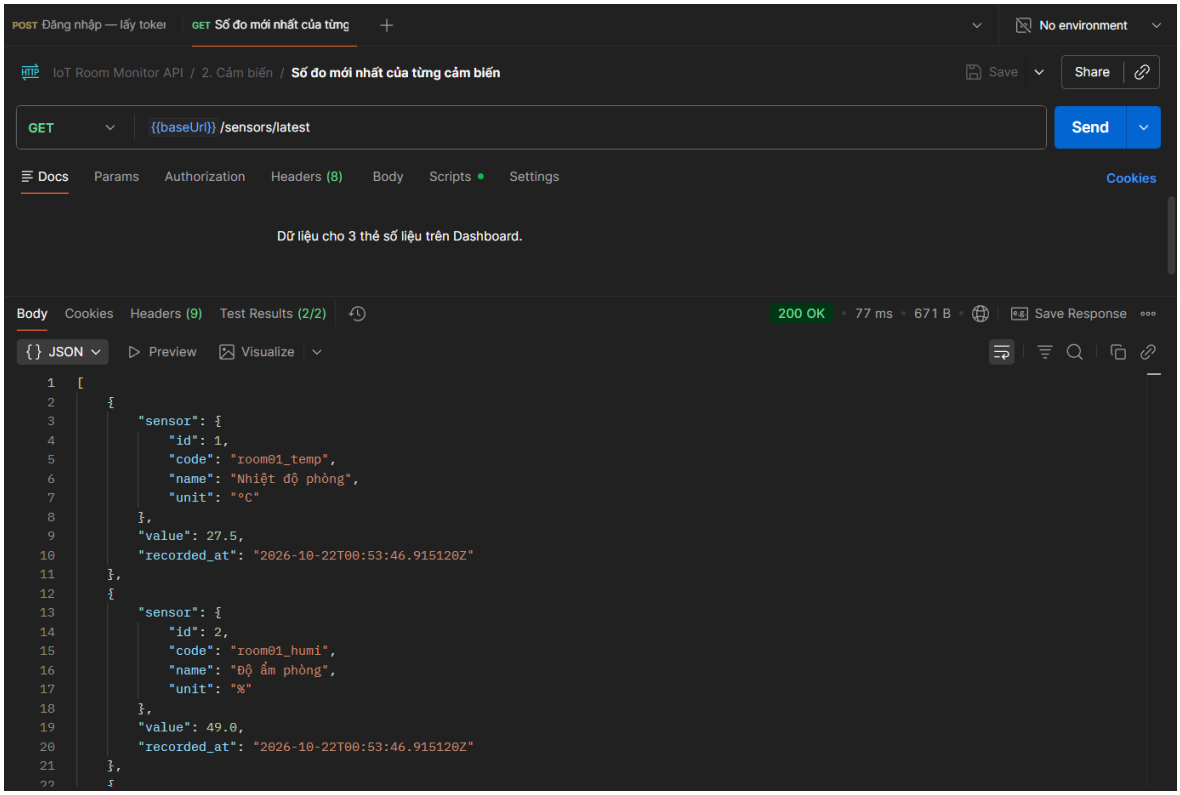
Hình 23. Tài liệu API trên Swagger UI

4.4 Kiểm thử API bằng Postman

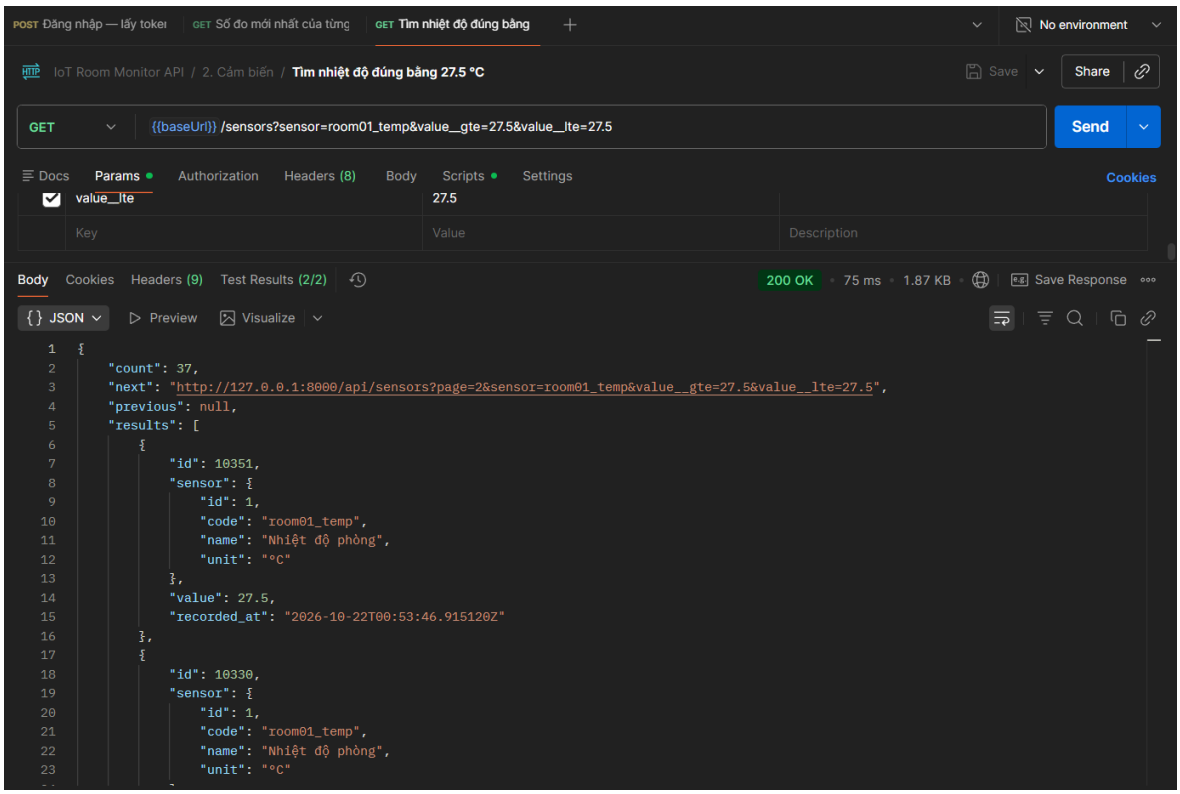
Các API của hệ thống được gom thành một collection Postman và chạy trên backend thật (daphne, cổng 8000). Collection tự đăng nhập rồi gắn token vào header Authorization của mọi request; mỗi request kèm script kiểm tra mã trạng thái và hình dạng dữ liệu trả về, kết quả hiện ở thẻ Test Results. Các hình dưới đây là những API chính theo thứ tự một phiên làm việc: đăng nhập, đọc số liệu, tra cứu có lọc, điều khiển thiết bị, xem lịch sử thao tác và một ca lỗi.



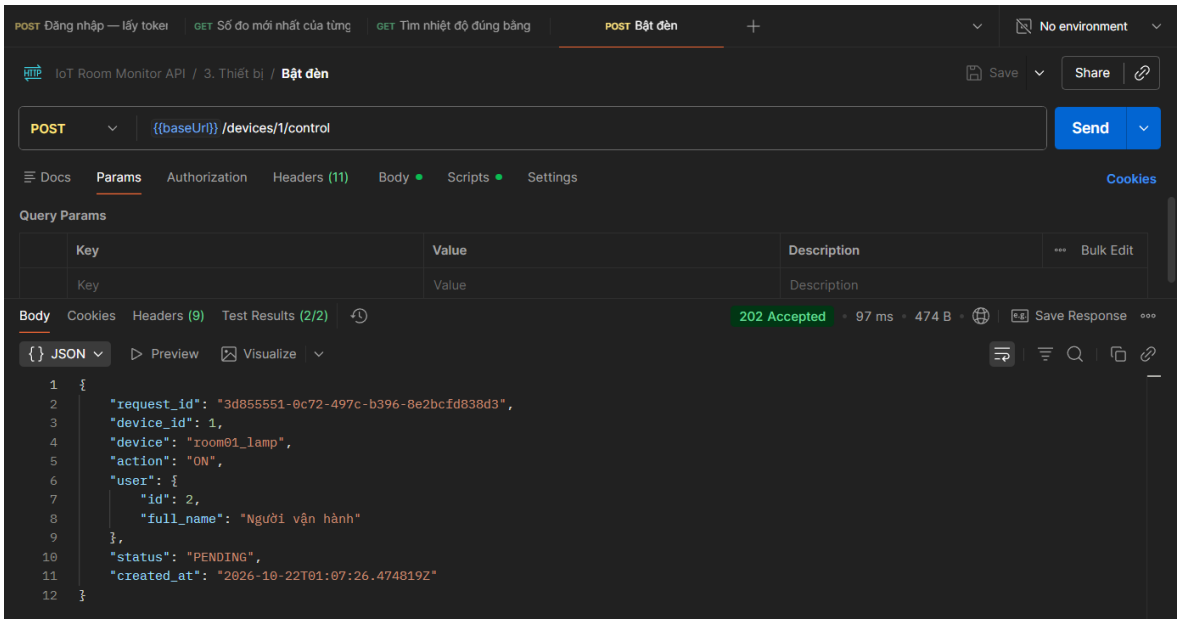
Hình 24. Đăng nhập và nhận token (POST /api/auth/login)



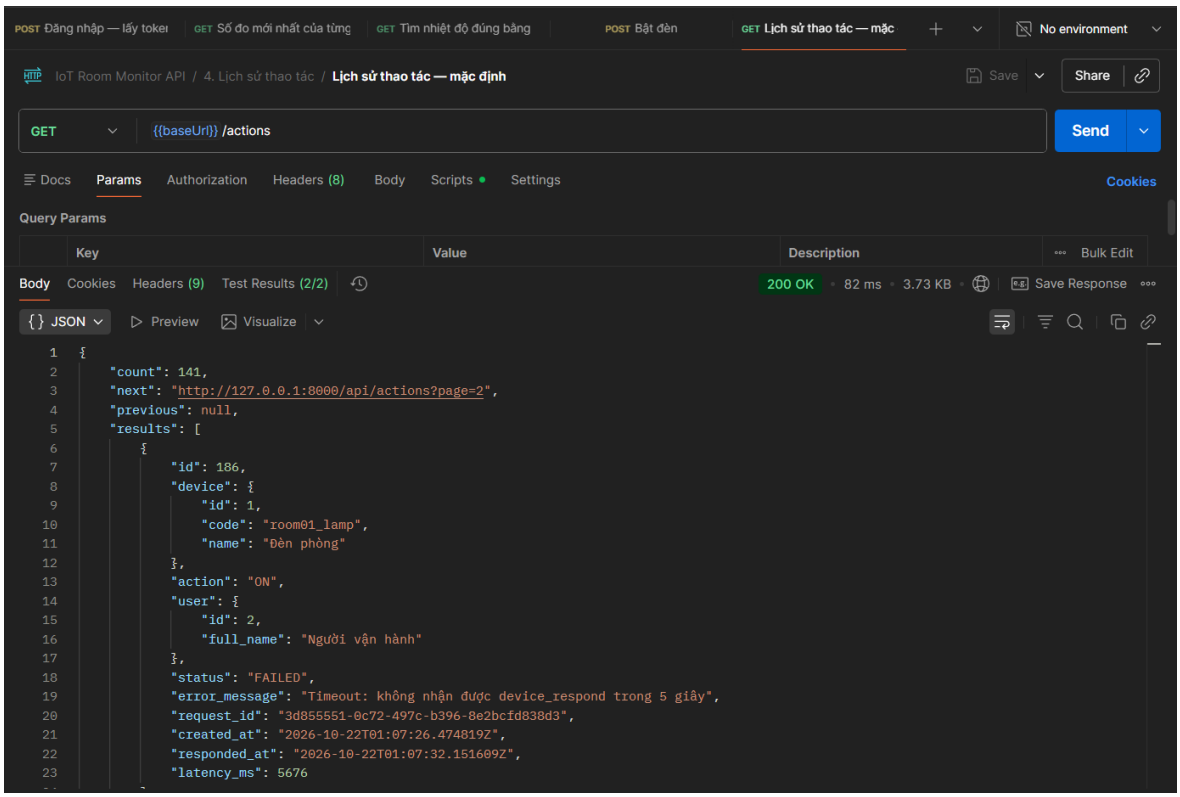
Hình 25. Số đo mới nhất của từng cảm biến (GET /api/sensors/latest)



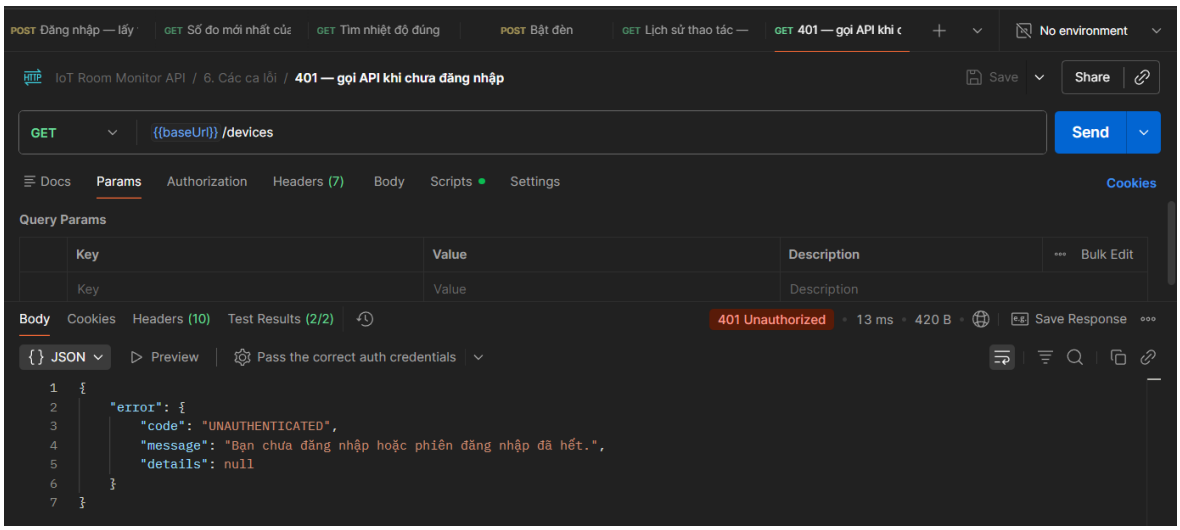
Hình 26. Tìm số đo nhiệt độ đúng bằng 27.5 °C, có phân trang (GET /api/sensors)



Hình 27. Gửi lệnh bật đèn, máy chủ trả 202 Accepted với trạng thái PENDING (POST /api/devices/1/control)



Hình 28. Lịch sử thao tác: lệnh bật đèn ở hình trước bị đánh FAILED do không có phản hồi từ thiết bị trong 5 giây (GET /api/actions)



Hình 29. Gọi API khi chưa đăng nhập bị từ chối với mã 401 UNAUTHENTICATED

4.5 Kết quả kiểm thử

Bảng 29. Kết quả kiểm thử

Hạng mục	Cách kiểm thử	Kết quả
Kiểm thử tự động backend	python manage.py test, mỗi ca mang mã kịch bản kiểm thử API	56/56 ca đạt
Kiểm thử đầu cuối	Daphne + worker + ESP8266 giả lập + Mosquitto + Redis	15/15 ca đạt (thiết bị phản hồi); 16/16 ca đạt (thiết bị không

Hạng mục	Cách kiểm thử	Kết quả
		phản hồi: FAILED sau 5,6 giây, trả 409 khi còn PENDING)
Kiểm thử giao diện	Kịch bản điều khiển Chrome tự động trên backend thật	Đăng nhập, chặn trang, biểu đồ, công tắc, lọc, phân trang, đăng xuất đều đúng; không có lỗi JavaScript
Tài liệu API	spectacular --validate --fail-on-warn	0 cảnh báo; đủ 10 endpoint chính, cộng 3 endpoint xem chi tiết theo id do Django REST Framework tự sinh (Hình 23)

Bảng 30. Kết quả đo hiệu năng

Chỉ tiêu	Yêu cầu	Đo được	Đánh giá
POST .../control (NFR-01)	≤ 2 s	70–100 ms	Đạt
Khứ hồi lệnh với mạch thật (NFR-01)	≤ 2 s	89–273 ms	Đạt
Từ lúc bấm công tắc tới lúc mở khoá trên giao diện	≤ 2 s	Khoảng 230 ms	Đạt
API danh sách (NFR-03)	≤ 500 ms	Khoảng 90 ms (lần đầu tải trang khoảng 390 ms)	Đạt
Phát hiện lệnh quá thời gian	5 s	5–6 s	Đúng thiết kế (vòng quét 1 giây)
Broker sai mật khẩu / sai cổng	Báo lỗi rõ	503 sau 27 ms / 2 s	Đạt

Đăng nhập mất khoảng 1–1,5 giây là có chủ đích: Django 5.2 băm mật khẩu bằng PBKDF2 với 1.000.000 vòng lặp, nhập sai mật khẩu cũng tốn đúng thời gian đó để chống dò mật khẩu và dò thời gian.

4.6 Các lỗi phát hiện và khắc phục

Bảng 31. Các lỗi phát hiện và cách khắc phục

#	Hiện tượng	Nguyên nhân	Cách khắc phục
1	Đèn đã bật nhưng lịch sử ghi FAILED	Thiết bị trả lời trước khi bản ghi PENDING được commit	Commit bản ghi trước rồi mới publish; broker lỗi thì xoá bản ghi
2	Mỗi lần gửi lệnh chậm khoảng 2 giây	localhost khiến Windows thử IPv6 trước, Mosquitto chỉ nghe IPv4	Dùng địa chỉ 127.0.0.1
3	WebSocket tự ngắt mỗi khoảng 11 giây khi mạch tắt	redis-py 8 mặc định hết giờ đọc 5 giây, trùng thời gian chờ của channels-redis	Đặt socket_timeout = 15 cho Redis, thêm kiểm thử chặn tái phát

#	Hiện tượng	Nguyên nhân	Cách khắc phục
4	Chu kỳ đầu sau khi cấp điện ra 0,9 °C / 0 %	DHT11 trả số rác ở lần đọc đầu, không phải NaN	Firmware đọc nháp và bỏ lần đọc đầu
5	Rút điện khi đèn bật: mạch tắt đèn nhưng giao diện vẫn ON	Backend không biết mạch đã khởi động lại	Mạch tự báo trạng thái (REPORT) mỗi lần kết nối lại broker

CHƯƠNG 5. KẾT LUẬN

5.1 Kết quả đạt được

Đề tài đã xây dựng hoàn chỉnh một hệ thống IoT giám sát và điều khiển môi trường phòng gồm đủ ba lớp: firmware ESP8266 thu thập số liệu và điều khiển thiết bị; backend Django xử lý, lưu trữ và cung cấp API; giao diện web React hiển thị số liệu thời gian thực, điều khiển thiết bị và tra cứu lịch sử. Hệ thống đã chạy thử thành công với mạch thật, các chỉ tiêu hiệu năng đều đạt yêu cầu đặt ra.

Các quyết định thiết kế chính:

- Tách backend thành hai tiến trình nối nhau qua Redis, phù hợp với bản chất đồng bộ của Django.
- Dùng request_id để ghép cặp lệnh với phản hồi; phát hiện lệnh quá thời gian bằng vòng quét dựa trên CSDL thay vì bộ hẹn giờ trong bộ nhớ.
- Lưu mỗi số đo thành một bản ghi riêng kèm danh mục cảm biến, giúp thêm cảm biến mới mà không đổi cấu trúc bảng.
- Chặn lệnh trùng ở cả hai tầng (giao diện và máy chủ), có cấu trúc lỗi thống nhất, có đăng nhập và ghi lại người thao tác.

Qua đề tài, sinh viên nắm được quy trình phát triển một sản phẩm IoT từ phân tích yêu cầu, thiết kế, lập trình đến kiểm thử, cũng như các vấn đề thực tế khi ghép nối phần cứng với phần mềm.

5.2 Hạn chế

Bảng 32. Hạn chế của hệ thống

#	Hạn chế	Ảnh hưởng
1	Chưa phân quyền theo vai trò	Mọi tài khoản đều điều khiển được thiết bị
2	Chỉ giám sát một phòng	Muốn thêm phòng phải bổ sung quản lý phòng và tách kênh WebSocket
3	Module LM393 đang dùng chỉ có ngõ ra số (DO)	Giá trị ánh sáng chưa liên tục, chưa đúng đơn vị lux
4	Cảm biến DHT11 sai số lớn (±2 °C, ±5 %RH)	Số liệu chỉ mang tính tham khảo
5	Thiết bị chấp hành mô phỏng bằng LED	Chưa điều khiển được tải điện thật
6	Chưa dùng Last Will của MQTT	Mạch mất điện phải chờ 30 giây giao diện mới báo ngoại tuyến
7	Chạy trên HTTP trong mạng nội bộ	Chưa đưa ra Internet an toàn được

5.3 Hướng phát triển

Bảng 33. Hướng phát triển

#	Hướng	Nội dung
1	Phân quyền	Dùng cột role sẵn có: VIEWER chỉ xem, OPERATOR được điều khiển, ADMIN quản trị

#	Hướng	Nội dung
2	Nhiều phòng	Thêm bảng phòng, tách nhóm WebSocket và topic MQTT theo phòng
3	Nâng cấp cảm biến	Dùng LM393 có ngõ ra analog (AO) hoặc BH1750; thay DHT11 bằng DHT22/SHT31
4	Cảnh báo	Gửi thông báo khi nhiệt độ, độ ẩm vượt ngưỡng
5	Thiết bị thật	Thay LED bằng module relay, bổ sung biện pháp an toàn điện
6	Triển khai	HTTPS/WSS, MQTT qua TLS, đóng gói bằng Docker Compose
7	Ứng dụng di động	Dùng lại toàn bộ API hiện có

TÀI LIỆU THAM KHẢO

[1] OASIS, *MQTT Version 3.1.1, OASIS Standard*, 2014.

[2] Aosong Electronics, *DHT11 Humidity & Temperature Sensor Datasheet*.

[3] Espressif Systems, *ESP8266EX Datasheet*.

[4] Django Software Foundation, *Django Documentation*, https://docs.djangoproject.com/

[5] Encode, *Django REST Framework Documentation*, https://www.django-rest-framework.org/

[6] Django Channels Project, *Channels Documentation*, https://channels.readthedocs.io/

[7] Eclipse Foundation, *Eclipse Mosquitto: An open source MQTT broker*, https://mosquitto.org/

[8] The PostgreSQL Global Development Group, *PostgreSQL Documentation*, https://www.postgresql.org/docs/

[9] Meta Open Source, *React Documentation*, https://react.dev/

[10] Joël Gähwiler, *arduino-mqtt*, https://github.com/256dpi/arduino-mqtt

[11] Nguyễn Quốc Uy, *Bài giảng học phần IoT và Ứng dụng*, Học viện Công nghệ Bưu chính Viễn thông, 2026.